



BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI
Work Integrated Learning Programs Division

**Post Graduate Program in Artificial Intelligence and Machine
Learning**

Fraud Detection in Financial Transactions

CAPSTONE PROJECT

Submitted in partial fulfillment of the requirements of the

Post Graduate Certification Program in Artificial Intelligence and Machine Learning

By

Khushboo Gupta	2023AIML093
Tanya Bansal	2023AIML062
Gaurav Balyan	2023AIML069
Subhadeep Roy	2023AIML137
Rakkiappan P	2023AIML086

Under the supervision of

PROF. SUDARSHAN S. DESHMUKH

Project work carried out at

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE Pilani (Rajasthan) INDIA

October, 2024

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI
SECOND SEMESTER 2023-24

Oct-2024 PCAMZC321 CAPSTONE PROJECT

Project Title: Fraud Detection in Financial Transactions

Name of Mentor: SUDARSHAN S. DESHMUKH

Name of Students:

- **Khushboo Gupta - 2023AIML093**
- **Tanya Bansal - 2023AIML062**
- **Gaurav Balyan - 2023AIML069**
- **Subhadeep Roy - 2023AIML137**
- **Rakkiappan P - 2023AIML086**

ACKNOWLEDGEMENT

We extend our profound appreciation to BITS, Pilani, for affording us the invaluable opportunity to undertake and successfully execute our project on an Artificial Intelligence and Machine Learning model.

Our gratitude extends to our esteemed mentor, **Prof. Sudarshan S. Deshmukh**, whose unwavering support, and expert guidance were instrumental in the timely and triumphant completion of our project. His steadfast assistance and encouragement served as a constant wellspring of motivation for our team.

Furthermore, we express our sincere thanks to all our distinguished professors, who imparted the fundamental and advanced principles of artificial intelligence and machine learning with exceptional nuance throughout this certification program.

In addition, we would like to convey our appreciation to our respective families and organizations for their unwavering support throughout this academic endeavor.

Khushboo Gupta (2023AIML093)

Tanya Bansal (2023AIML062)

Gaurav Balyan (2023AIML069)

Subhadeep Roy (2023AIML137)

Rakkiappan P (2023AIML086)

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI

CERTIFICATE

This is to certify that the Capstone Project entitled "FRAUD DETECTION IN FINANCIAL TRANSACITONS" and submitted by Ms. Khushboo Gupta (2023AIML093), Ms. Tanya Bansal (2023AIML062), Mr. Gaurav Balyan (2023AIML069), Mr. Subhadeep Roy (2023AIML137) and Rakkiappan P (2023AIML086) in partial fulfillment of the requirements of PCAM ZC321 Capstone Project embodies the work done by him/her under my supervision.

Place : BITS Pilani

Signature of the mentor

Sudashan Deshmukh

Date:

Name: Sudarshan S. Deshmukh

ABSTRACT

Fraudulent activities in financial transactions present a significant challenge to financial institutions, resulting in substantial monetary losses and undermining consumer trust. The increasing complexity and volume of financial data call for advanced and scalable fraud detection systems capable of identifying suspicious patterns in real-time. This dissertation explores the development and application of machine learning techniques for detecting fraud in financial transactions, with a focus on both supervised and unsupervised learning approaches.

The study begins by examining the limitations of traditional rule-based systems, which often struggle to adapt to evolving fraud tactics and large-scale data. A key focus is on leveraging clustering algorithms, such as K-means, to detect anomalies that deviate from typical transaction behaviors, thus highlighting potential fraudulent activities. Additionally, classification models, including logistic regression, decision trees, and neural networks, are employed to improve detection accuracy by learning from historical labeled data.

To address the issue of imbalanced datasets—common in fraud detection—various resampling techniques, such as Synthetic Minority Over-sampling Technique (SMOTE), are applied to enhance model performance. The research also incorporates ensemble learning methods to improve the robustness of fraud detection systems by combining multiple models. Furthermore, this dissertation evaluates real-time fraud detection systems using stream processing and online learning techniques, ensuring that models can adapt to new fraud patterns quickly.

The proposed methods are tested on the provided financial transaction dataset, and the results are compared in terms of detection accuracy, precision, recall, and computational efficiency. The findings demonstrate that a hybrid approach, combining clustering for anomaly detection and classification for precise identification, offers a powerful solution for financial fraud detection. This work contributes to the ongoing efforts to enhance fraud prevention systems and provides a foundation for future research in adaptive and scalable machine learning solutions for financial security.

TABLE OF CONTENTS

1.PROBLEM STATEMENT	8
2.PROJECT OBJECTIVE	9
3.LITERATURE REVIEW	10
4.MACHINE LEARNING PROCESS FLOW	13
5.PEOPLE, HARDWARE AND SOFTWARE RESOURCES.....	14
6.POTENTIAL DATA CHALLENGES AND RISKS.....	15
6.1 Challenges.....	15
6.2 Risks.....	16
6.3 Mitigation Strategies	16
7.PLAN OF WORK.....	17
8.PRE-PROCESSING STEPS.....	19
8.1 Data Loading and Field Summary	19
8.2 Exploratory Data Analysis	19
8.3 Feature Engineering and Data Preprocessing.....	20
8.4 Handling Imbalanced classes and Missing Fraud Data	22
8.5 Scaling and Normalization	23
8.6 Visualization for Data Summarization.....	24
9. ML MODELLING AND TECHNIQUES APPLIED	25
10. CODE & SCREENSHOTS	28
11. INTERPRETATION	67
11.1 Justification of Measures / Metrics used.....	67
11.2 Project output in terms of above measures/metrics.....	67
11.3 Visualization Plots	71
12. FUTURE WORK & EXTENSION OR SCOPE OF IMPROVEMENTS	76
CONCLUSION/ RECOMMENDATIONS	78
DIRECTIONS FOR FUTURE WORK	79
BIBLIOGRAPHY/ REFERENCES	80
DULY COMPLETED CHECKLIST	81

CHAPTER 1

PROBLEM STATEMENT

Develop a machine learning model to detect fraudulent transactions in real-time, enhancing the security of financial systems.

- Fraudulent activities in the financial sector have far-reaching consequences, affecting both businesses and consumers.
- As digital transactions become more prevalent, the potential for fraud grows, making it imperative to have systems in place that can detect and mitigate fraudulent transactions in real time
- **Example:** Consider a large e-commerce platform handling millions of credit card transactions daily. Fraudsters might attempt to make unauthorized purchases by using stolen card details. Without real-time fraud detection, such transactions would go unnoticed until the cardholder or bank identifies them—by which time it might be too late to reverse the damage.
- Developing a machine learning model to detect fraudulent transactions is not just a reactive measure but a proactive step toward building a more secure financial ecosystem.
- By leveraging real-time data analysis and pattern recognition, financial institutions can mitigate risks, protect their customers, and stay ahead of evolving fraud tactics.

CHAPTER 2

PROJECT OBJECTIVE

The objective of this project is to develop a machine learning model that detects fraudulent financial transactions in real-time, enhancing the security and reliability of financial systems. The model aims to:

1. **Identify and Mitigate Fraud in Real-Time:** Detect suspicious patterns and behaviors in transaction data as they occur, minimizing the time between fraud attempts and their detection.
2. **Enhance Transaction Security:** Safeguard consumers and businesses by reducing the risk of unauthorized transactions, ensuring the integrity of digital financial systems.
3. **Adapt to Evolving Fraud Tactics:** Continuously learn from new data to recognize emerging fraud strategies, making the model robust and adaptive to evolving threats.
4. **Minimize False Positives and Negatives:** Achieve high detection accuracy while maintaining low false-positive rates to prevent legitimate transactions from being flagged, thus improving customer experience and operational efficiency.

By achieving these objectives, the project seeks to provide a scalable, efficient, and proactive solution to combat financial fraud in increasingly digitized financial ecosystems.

CHAPTER 3

LITERATURE REVIEW

Fraud detection in financial transactions has been a significant area of research due to the increasing complexity of fraudulent schemes and the growth of online and digital financial activities. A wide array of techniques, ranging from traditional rule-based systems to modern machine learning and deep learning approaches, has been explored and developed to tackle the problem of detecting and preventing fraud. Below is a summary of the key contributions and previous work done in this area.

- **Traditional Approaches:** Early fraud detection systems primarily relied on **rule-based methods**, where specific conditions or thresholds are predefined to flag fraudulent activities. These systems worked well for detecting simple and repetitive fraud patterns. However, they often struggled with adaptability and scalability. As fraud tactics evolved, rule-based systems became increasingly inefficient due to their inability to learn from data or detect complex fraud patterns. Notable works in this area include the application of **expert systems** and **heuristic-based techniques** (Bolton & Hand, 2002), which were limited by their dependence on static rules and high false-positive rates.
- **Supervised Machine Learning Approaches:** The shift towards data-driven methods led to the exploration of **supervised learning techniques** in fraud detection. These methods require labeled data, where historical transaction records are used to train models to classify transactions as fraudulent or non-fraudulent. Popular supervised models include:
 - **Logistic Regression:** Widely used due to its interpretability and simplicity. Researchers such as Maes et al. (2002) have demonstrated its application in financial fraud detection.
 - **Decision Trees and Random Forests:** These algorithms gained popularity due to their ability to handle non-linear relationships in transaction data. **Random Forests** are especially noted for their robustness, as demonstrated by Whitrow et al. (2009), who applied the technique to detect credit card fraud.
 - **Support Vector Machines (SVMs):** SVMs have been used to detect fraud by creating decision boundaries between fraudulent and non-fraudulent transactions. Bhattacharyya et al. (2011) used SVMs to demonstrate competitive accuracy in fraud detection.
 - Although supervised models provide good performance, they are heavily dependent on labeled datasets, which can be scarce or imbalanced, as fraud transactions make up a small portion of overall data.
- **Unsupervised Learning and Anomaly Detection:** Unsupervised learning techniques, particularly **anomaly detection algorithms**, have become prominent in fraud detection due to their ability to identify unusual patterns without relying on labeled data. Fraudulent transactions are often rare and exhibit behaviors that deviate from normal transaction patterns. Key unsupervised approaches include:
 - **K-means Clustering:** Utilized for anomaly detection by identifying clusters of normal transactions, with outliers flagged as potential fraud (Jha et al., 2012). This method works well for high-dimensional financial data.
 - **Autoencoders and Neural Networks:** Recent work by Ngai et al. (2011) has explored the use of autoencoders in detecting anomalies in transactional data. These neural networks learn a compressed representation of data and can effectively identify abnormal patterns.

- **Isolation Forests:** This algorithm isolates anomalies by randomly partitioning the data. Liu et al. (2008) applied isolation forests in fraud detection, achieving high performance in spotting outlier transactions.
 - **Deep Learning Approaches:** With the growth of computational power and the availability of large-scale transaction data, **deep learning models** have shown promise in fraud detection. **Recurrent Neural Networks (RNNs)** and **Long Short-Term Memory (LSTM) networks** have been particularly effective in learning temporal patterns in transaction sequences. In 2017, Wei et al. demonstrated the power of LSTMs in detecting fraud by analyzing sequential dependencies in transaction records.
- Moreover, **Convolutional Neural Networks (CNNs)**, traditionally used for image processing, have also been adapted for fraud detection, particularly for analyzing structured data in a grid-like format (Fiore et al., 2019).
- **Ensemble Learning:** Ensemble methods such as **XGBoost**, **Bagging**, and **Boosting** have also been explored to combine the strengths of multiple classifiers and improve detection accuracy. Combining models, as done by Phua et al. (2010), provides more robust solutions, reducing both false positives and negatives. These methods can take advantage of both linear and non-linear patterns in transactional data.
 - **Handling Imbalanced Data:** One of the critical challenges in fraud detection is the highly imbalanced nature of the dataset, where fraudulent transactions are rare compared to legitimate ones. **Synthetic Minority Over-sampling Technique (SMOTE)**, as proposed by Chawla et al. (2002), is commonly used to address this issue by generating synthetic examples of the minority class. **Cost-sensitive learning** has also been adopted to assign higher misclassification costs to fraudulent transactions (Dal Pozzolo et al., 2014).
 - **Real-Time Fraud Detection:** The demand for real-time fraud detection has led to the exploration of **streaming data models** and **online learning algorithms**. These approaches, such as those presented by Zliobaite et al. (2012), focus on processing transactions in a continuous, time-sensitive manner, enabling immediate identification of fraudulent activities. **Apache Kafka** and **Apache Flink** are commonly used platforms for building real-time fraud detection pipelines that can handle large volumes of transactional data.
 - **Recent Developments and Future Directions:** Recent works emphasize the importance of hybrid models that combine anomaly detection with supervised classification techniques to improve both the accuracy and adaptability of fraud detection systems (Zhang et al., 2020). Additionally, the integration of explainability in fraud detection models, such as using **SHAP** or **LIME** to interpret predictions, is becoming increasingly important for regulatory compliance and operational transparency.

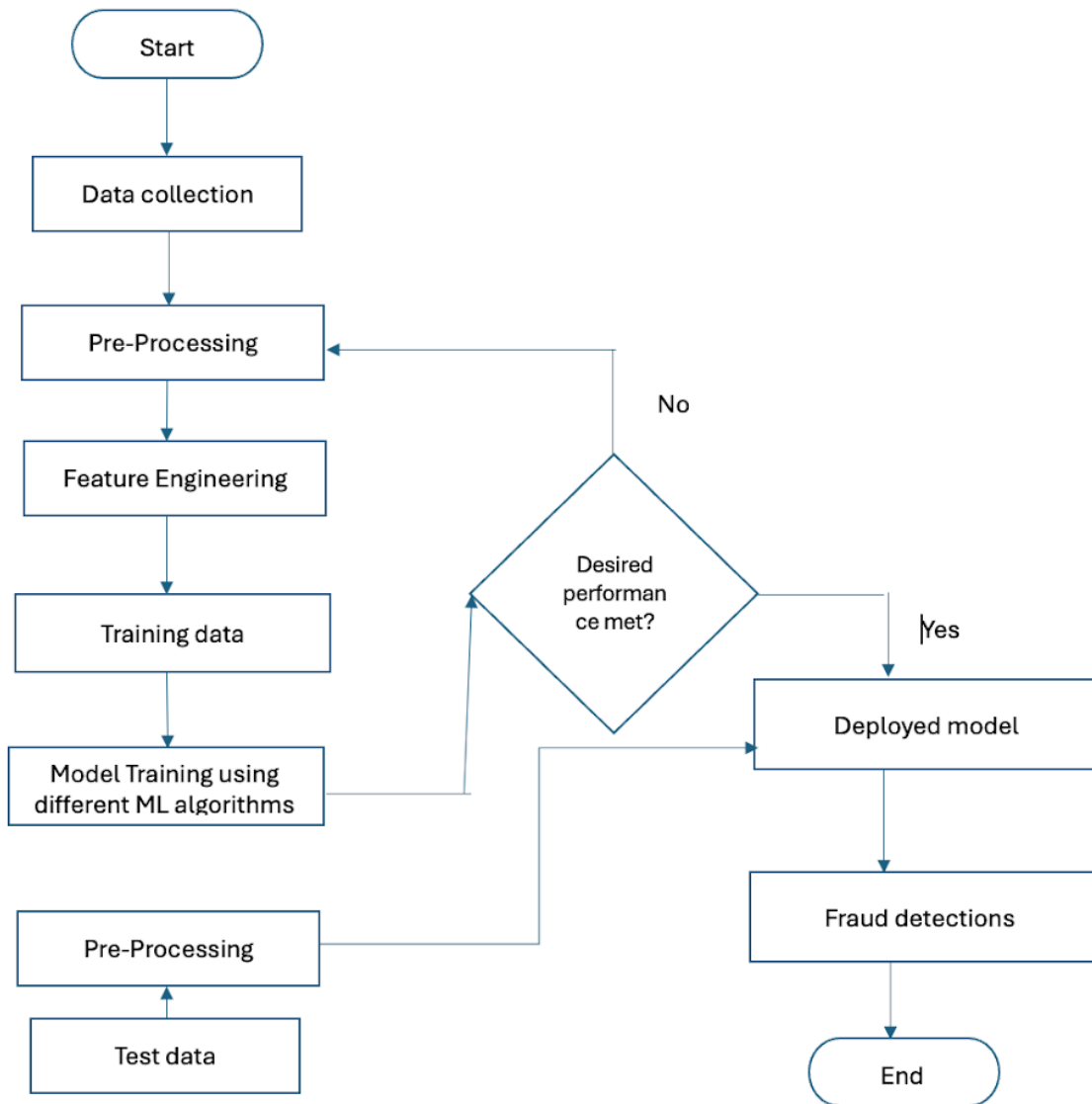
The rise of **graph-based methods** for analyzing relationships between entities in transaction data is another growing area of research. Graph neural networks (GNNs) have been proposed to detect fraud in transaction networks by identifying suspicious clusters of entities (Dou et al., 2020).

The literature highlights the evolution of fraud detection techniques, from traditional rule-based systems to advanced machine learning models capable of real-time fraud detection. While significant progress has been made, future work will likely focus on the scalability of detection systems, the integration of diverse data sources, and the development of adaptive models that can continuously learn from new fraud patterns.

This review provides a comprehensive overview of previous work in fraud detection and establishes a foundation for developing cutting-edge machine learning models that address current challenges in the field.

CHAPTER 4

MACHINE LEARNING PROCESS FLOW:



The ML process includes gathering the data from source provided and preprocessing steps like cleaning, transform and prepare the data for analysis (handling missing values, scaling and encoding). We created the new features mentioned in report. After that choosing the appropriate classification model and then use the prepared data to train the model. After that assess the model's performance by using different parameters like F1-score. Optimize the model through hyperparameter tuning. The process is iterative and requires refinements for better results.

CHAPTER 5

PEOPLE, HARDWARE AND SOFTWARE RESOURCES

5.1 People: This project was developed by a team of five members, including a project guide.

- **Mentor:** Prof. Sudarshan S. Deshmukh
- **Team:**
 - Khushboo Gupta (2023AIML093)
 - Tanya Bansal (2023AIML062)
 - Gaurav Balyan (2023AIML069)
 - Subhadeep Roy (2023AIML137)
 - Rakkiappan P (2023AIML086)

5.2 Hardware Resources: The hardware used for this project is listed below:

- Model Name: Lenovo thinkpad
- Model Identifier: Think pad
- Model Number: 20I1S16J00
- Chip: Core i5
- Total Number of Cores: 12 (6 performance and 6 efficiency)
- Memory: 16 GB

Operating System:

The code has been tested on Windows 11

5.3 Software Resources: The project was developed using Python programming language and the following libraries:

- **Scikit Learn APIs:** Libraries used for preprocessing the dataset.
- **PyTorch:** torch, torch.nn.parallel, torch. Distributed - Parallel processing and multitasking

Softwares Required: Anaconda, Jupyter Notebook, Google Colab, Pycharm, Git Hub, Fast API

CHAPTER 6

POTENTIAL DATA CHALLENGES AND RISKS

When developing a machine learning model for real-time fraud detection in financial transactions, several data-related challenges and risks can arise. These challenges must be carefully addressed to ensure that the model is accurate, reliable, and scalable. Below are the potential data challenges and risks:

6.1 Challenge:

- **Imbalanced Dataset:** Fraudulent transactions typically make up a tiny fraction of the total dataset, often less than 1%. This imbalance can lead to models being biased toward predicting legitimate transactions because the majority of the data is non-fraudulent.
- **Lack of Labeled Data for Fraud:** Labeled fraudulent transactions are rare, and it may be difficult to obtain enough positive samples (fraud cases) for training a supervised machine learning model.
- **Data Privacy and Compliance:** Financial transaction data is highly sensitive and subject to privacy regulations like GDPR, CCPA, or PCI DSS. Collecting, processing, and storing data for model training must comply with these regulations.
- **Data Quality and Consistency:** Transaction data might contain errors, missing values, duplicates, or inconsistencies due to issues in data logging or processing systems. Fraudulent behaviors may also evolve over time, leading to concept drift (i.e., changing fraud patterns).
- **Feature Selection and Engineering:** Transaction data is complex, and meaningful features that can distinguish fraudulent from legitimate transactions are not always obvious. Moreover, some features might introduce bias or noise.
- **Dynamic and Evolving Fraud Patterns:** Fraudsters continually adapt their techniques, meaning that the types of fraud and the associated patterns evolve over time.
- **Latency and Scalability in Real-Time Detection:** Real-time fraud detection requires models that can process large volumes of transactions quickly and accurately. Latency and computational resource limitations could hinder real-time performance.
- **Interpretability and Explainability:** Advanced machine learning models, such as deep learning, may achieve high accuracy but often lack transparency and interpretability, especially in complex decision-making like fraud detection.
- **Integration with Legacy Systems:** Financial institutions may have existing fraud detection systems, and the new machine learning model must integrate seamlessly with legacy infrastructure.
- **False Positives and Customer Experience:** A model that is overly aggressive in detecting fraud may flag too many legitimate transactions as fraudulent, leading to false positives.

6.2 Risk:

- **Imbalanced Dataset:** The model may have high accuracy but fail to detect fraudulent activities (high false negatives), or it may flag too many legitimate transactions as fraud (high false positives), resulting in poor customer experience.
- **Lack of Labeled Data for Fraud:** Inadequate labeled data can lead to poor model performance, especially in capturing the diversity of fraud patterns.
- **Data Privacy and Compliance:** Mismanagement of sensitive data could result in legal consequences, fines, and reputational damage. It also poses ethical risks regarding customer privacy
- **Data Quality and Consistency:** Poor data quality can degrade model performance, and inconsistent data could cause the model to produce unreliable results. Concept drift can make the model obsolete over time.
- **Feature Selection and Engineering:** Poor feature selection can result in suboptimal model performance, or worse, introduce bias, leading to incorrect fraud detection
- **Dynamic and Evolving Fraud Patterns:** A static model may become ineffective over time, leading to undetected fraud or frequent false positives.
- **Latency and Scalability in Real-Time Detection:** High-latency models could fail to detect fraud in real-time, resulting in financial losses. Moreover, scaling the model to handle millions of transactions efficiently is complex.
- **Interpretability and Explainability:** Regulatory bodies, banks, or consumers may require clear explanations of why a particular transaction was flagged as fraudulent. Lack of interpretability can reduce trust in the system.
- **Integration with Legacy Systems:** Incompatibilities with existing systems could lead to operational inefficiencies or even disruptions in the detection process.
- **False Positives and Customer Experience:** Excessive false positives can frustrate customers, resulting in a poor user experience, customer attrition, and potential loss of trust in the system.

6.3 Mitigation Strategies:

- **Imbalanced Dataset:** Techniques like Synthetic Minority Over-sampling (SMOTE), cost-sensitive learning, or ensemble methods can help handle this imbalance, though care must be taken to avoid overfitting to synthetic data.
- **Lack of Labeled Data for Fraud:** Unsupervised or semi-supervised learning approaches, such as anomaly detection techniques, can be employed. Alternatively, transfer learning from other fraud detection datasets could help supplement data deficiencies.
- **Data Privacy and Compliance:** Ensure proper anonymization or pseudonymization of sensitive data, apply encryption for data at rest and in transit, and implement strict access controls. Use privacy-preserving machine learning techniques such as differential privacy if necessary.
- **Data Quality and Consistency:** Implement rigorous data cleaning processes,

including outlier detection and imputation techniques for missing data. Continuous monitoring and retraining of the model are crucial to address concept drift and maintain performance.

- **Feature Selection and Engineering:** Conduct thorough exploratory data analysis (EDA) to identify relevant features, and use domain expertise to select or engineer meaningful features. Advanced feature selection techniques, such as Recursive Feature Elimination (RFE) or automated feature engineering, can help improve feature quality.
- **Dynamic and Evolving Fraud Patterns:** Implement online learning or incremental learning techniques to allow the model to adapt to new patterns in real time. Regular retraining and monitoring of the model performance should be part of the deployment process.
- **Latency and Scalability in Real-Time Detection:** Optimize models for speed and efficiency, potentially using streaming data platforms such as Apache Kafka or Apache Flink. Employ lightweight models or techniques like approximate nearest neighbor search for anomaly detection in real-time.
- **Interpretability and Explainability:** Regulatory bodies, banks, or consumers may require clear explanations of why a particular transaction was flagged as fraudulent. Lack of interpretability can reduce trust in the system.
- **Integration with Legacy Systems:** Ensure that the new model is compatible with existing systems by using standardized APIs and flexible deployment methods (e.g., cloud-based, on-premise). Collaborate with IT and operational teams to ensure smooth integration.
- **False Positives and Customer Experience:** Strike a balance between precision and recall by fine-tuning the model's thresholds, leveraging ensemble models to reduce error, and integrating additional features (such as customer profiles or transaction histories) to increase accuracy. Implement feedback loops where flagged transactions can be reviewed manually and fed back into the system for continuous learning.

CHAPTER 7

PLAN OF WORK

Sr No	Milestones	24-Aug	31-Aug	07-Sep	14-Sep	21-Sep	28-Sep	05-Oct	12-Oct
1	Scoping and Banking Industry Insights								
1.1	Exploring the data								
1.2	Analyzing the features								
1.3	Creating relationships								
2	Dataset Exploration and Data Pre-processing								
2.1	Preprocessing data (missing values, duplicate, null values)								
2.2	Finding the correlation								
2.3	Analyzing the types of fraud available								
3	Research Machine Learning Models								
3.1	Applied feature engineering , adding new columns								
3.2	Scaling and normalizing the data								
3.3	Label and frequency encoding on the data								
4	Design the Fraud Detection Model								
4.1	Different strategies to handle Class imbalance issue								
4.2	Apply on few algorithms to check the performance								
4.3	Try out few algorithm for first iteration								
5	Implement and train the models								
5.1	Multiple modelling techniques to test the performance								
5.2	Applying SMOTE to handle class imbalance problem								
5.3	Fine tune model								
6	Test the Models with simulated Scenarios								
6.1	Fine-tuning Based on Feedback and Additional Data								
6.2	Thorough Testing, Validation								
6.3	Documentation of the Final Model and Deployment Preparation								
7	Write Final Report, project presentation, Future Work								
	Create Github, fast API , report								



Weekly checkpoint with mentor



Review and viva with faculty

CHAPTER 8

PRE-PROCESSING STEPS

8.1 Data Loading and Field Summary

The dataset consists of synthetic financial transaction records. The data was loaded using the pandas library, and a summary of the fields is provided below:

Field	Description
Step	Represents time in hours (31 days = 744 steps).
type	Type of transaction (CASH-IN, CASH-OUT, DEBIT, PAYMENT, TRANSFER).
amount	Transaction amount in the local currency.
nameOrig	ID of the customer initiating the transaction.
oldbalanceOrg	Balance of the customer before the transaction.
newbalanceOrig	Balance of the customer after the transaction.
nameDest	ID of the transaction recipient.
oldbalanceDest	Balance of the recipient before the transaction.
newbalanceDest	Balance of the recipient after the transaction.
isFraud	Binary flag indicating whether the transaction is fraudulent.
isFlaggedFraud	Binary flag indicating whether the transaction was flagged as potentially fraudulent.

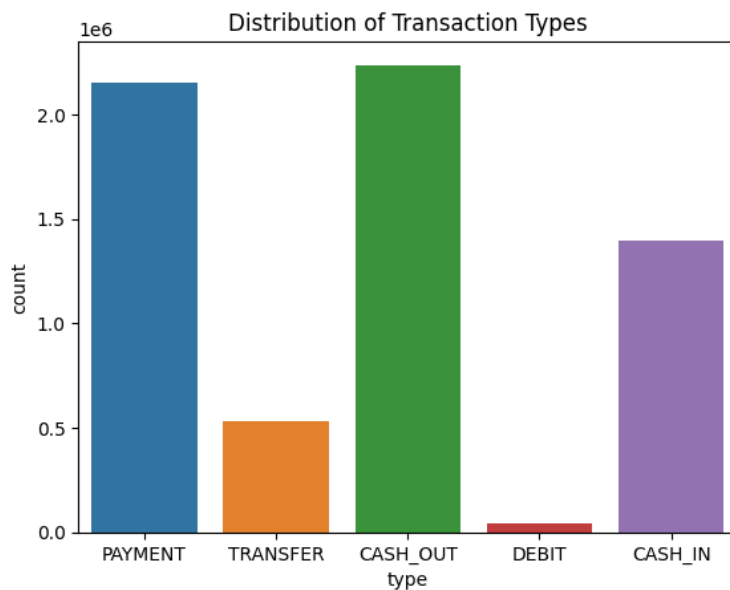
8.2 Exploratory Data Analysis (EDA)

8.2.1 Data Overview

- Basic summary statistics were calculated using `data.describe()`, and missing values were checked with `data.isnull().sum()`. No significant missing data were found.
- The dataset was inspected using `data.head()` to review the structure and nature of transactions.

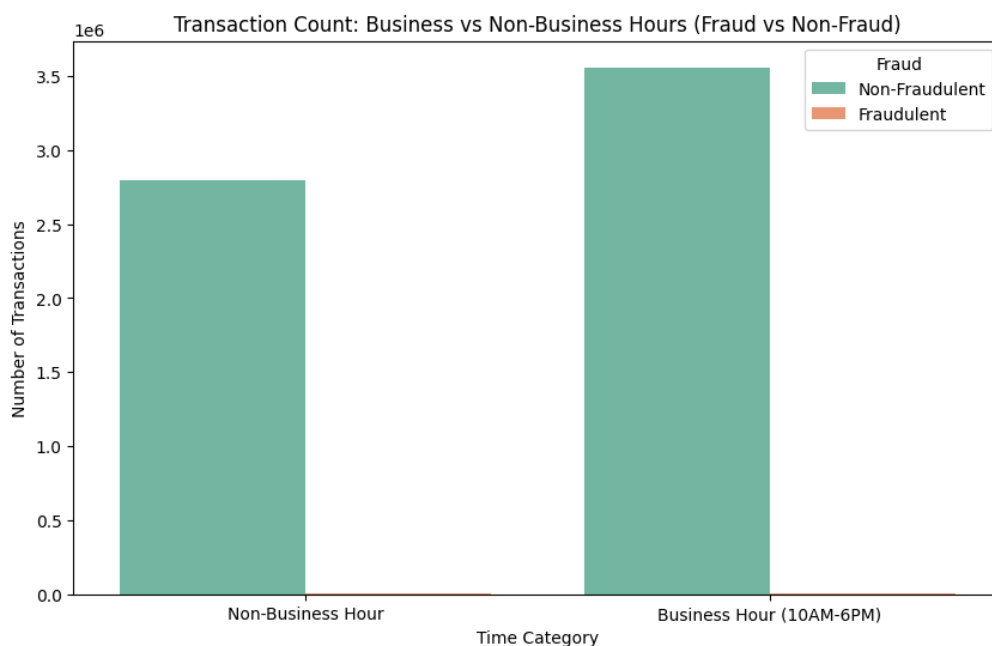
8.2.2 Transaction Type Distribution

- The dataset contains five transaction types: CASH-IN, CASH-OUT, DEBIT, PAYMENT, and TRANSFER.
- The distribution of transaction types was visualized using a count plot, highlighting that CASH-OUT and PAYMENT are the most common types.



8.2.3 Business vs Non-Business Hours

- Transactions were categorized into business (10 AM - 6 PM) and non-business hours. Fraudulent transactions were more frequent during non-business hours.
- A count plot showed the number of fraudulent and non-fraudulent transactions during business and non-business hours.



8.3 Feature Engineering and Data Preprocessing

8.3.1 Feature Engineering

Several new features were created to provide additional insights for fraud detection:

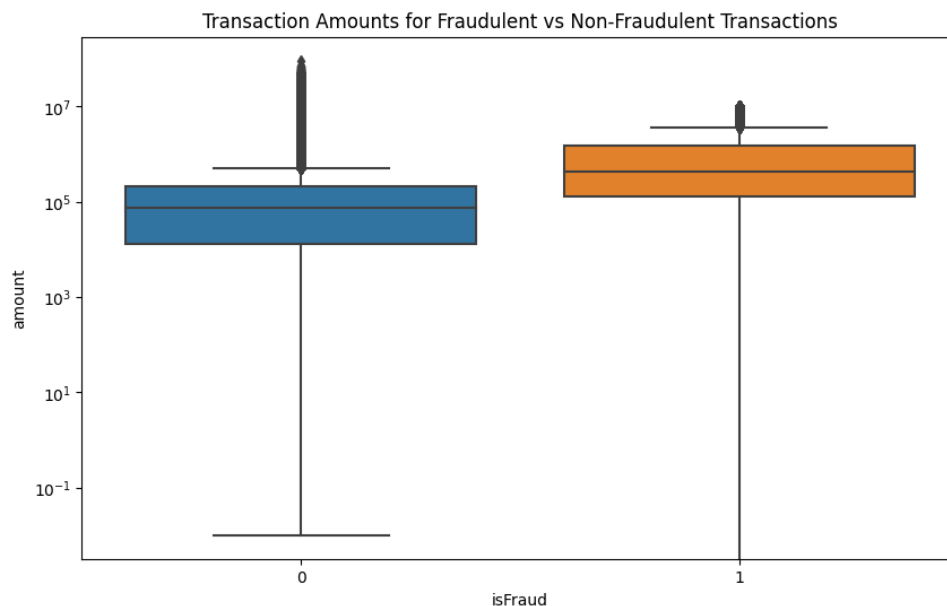
- **Balance Difference:** `balance_diff_orig` and `balance_diff_dest` were calculated to capture changes in balance before and after transactions.

- **Transaction-to-Balance Ratio:** `amount_to_oldbalance_ratio` was computed to identify large transactions relative to the customer's original balance. High ratios may indicate suspicious activities.
- **Balance Decrease Ratio:** `balance_decrease_ratio` was added to compare the decrease in balance relative to the transaction amount.

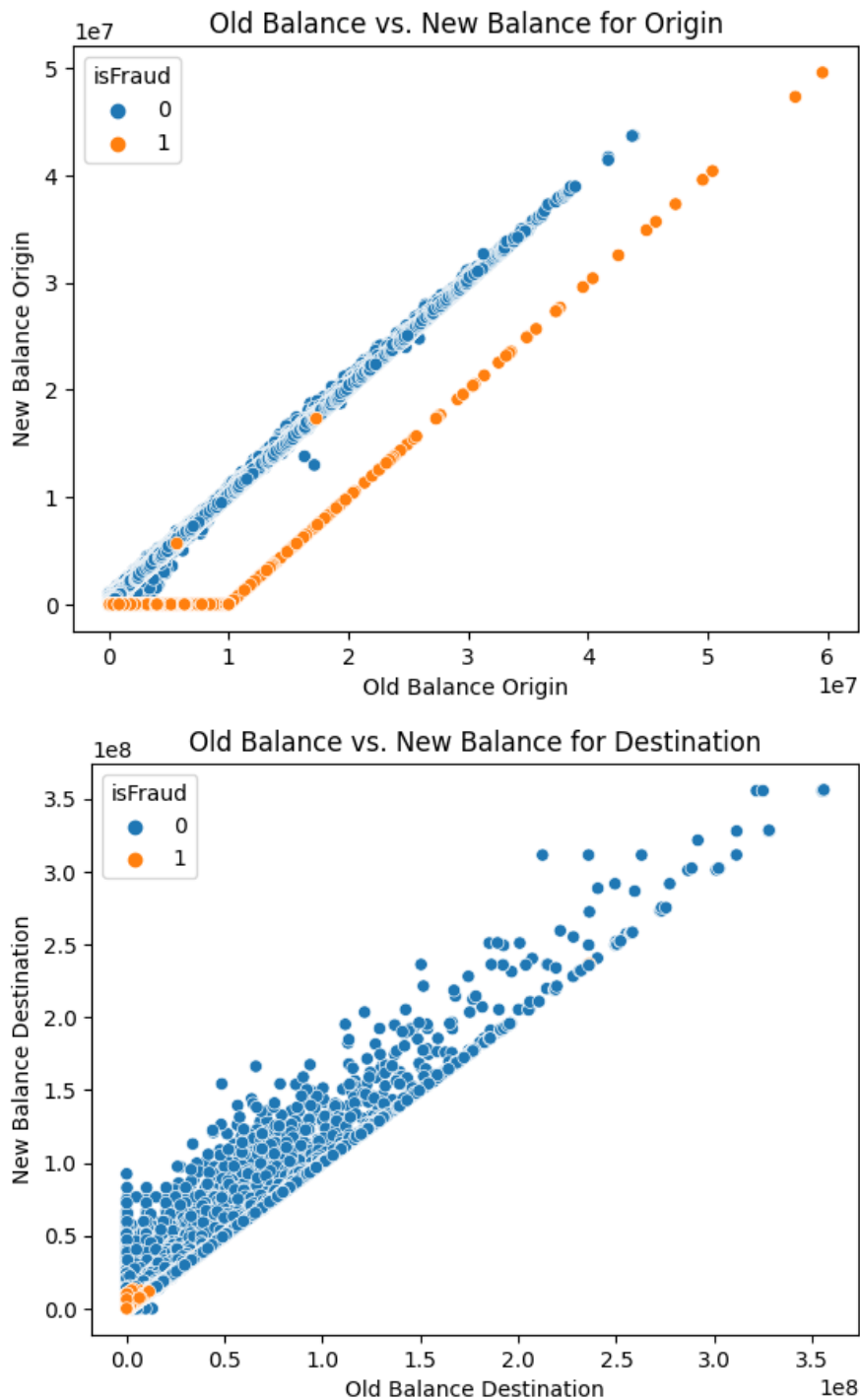
8.3.2 Outlier Detection and Visualization

Visualizing transaction amounts and balances helped identify potential outliers, particularly for fraud detection:

- **Histogram Visualization:** Transaction amounts, old balances, and new balances were plotted to observe the distribution.
- **Box Plots:** A log-scaled box plot was used to compare the transaction amounts between fraudulent and non-fraudulent transactions.



- **Scatter Plots:** Visualized relationships between `oldbalanceOrig` and `newbalanceOrig` and between `oldbalanceDest` and `newbalanceDest` for both fraudulent and non-fraudulent transactions.



8.4 Handling Imbalanced Classes and Missing Fraud Data

8.4.1 Synthetic Data Generation for Missing Fraudulent Transactions

Certain transaction types, such as DEBIT, CASH-IN, and PAYMENT, lacked fraudulent cases. To address this imbalance:

- **Clustering:** KMeans clustering was applied to the fraudulent transactions of CASH-OUT and TRANSFER to identify patterns in fraud behavior.

- **Synthetic Fraud Generation:** Based on the fraud clusters, synthetic fraudulent transactions were generated for DEBIT, CASH-IN, and PAYMENT types, ensuring the synthetic data followed the distribution patterns of real fraudulent cases.

8.4.2 Label Encoding

Categorical variables like `nameOrig` and `nameDest` were frequency-encoded, and transaction types were label-encoded for compatibility with machine learning models.

8.5 Scaling and Normalization

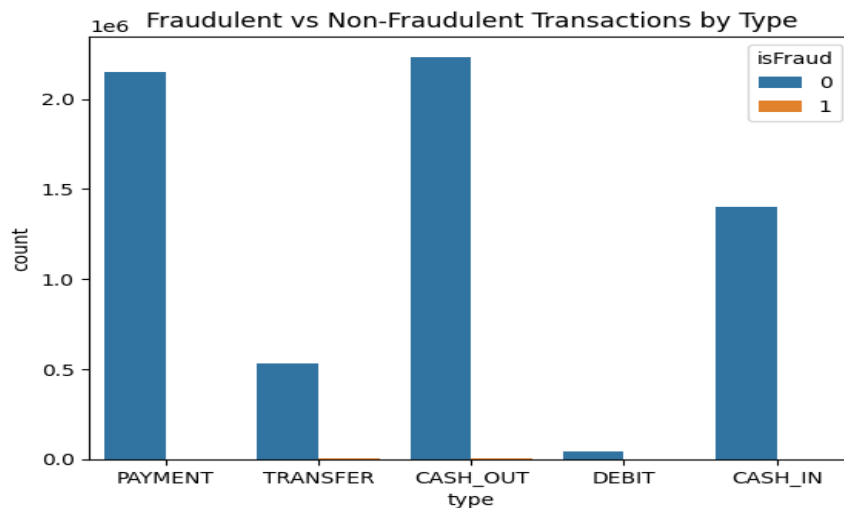
To ensure that features are on a consistent scale for machine learning algorithms:

- **Standard Scaling:** Numerical features (`step`, `amount`, `oldbalanceOrig`, `newbalanceOrig`, `oldbalanceDest`, `newbalanceDest`) were scaled using the `StandardScaler`.
- **One-Hot Encoding:** The `type` column (transaction types) was one-hot encoded, resulting in separate binary columns for each transaction type.

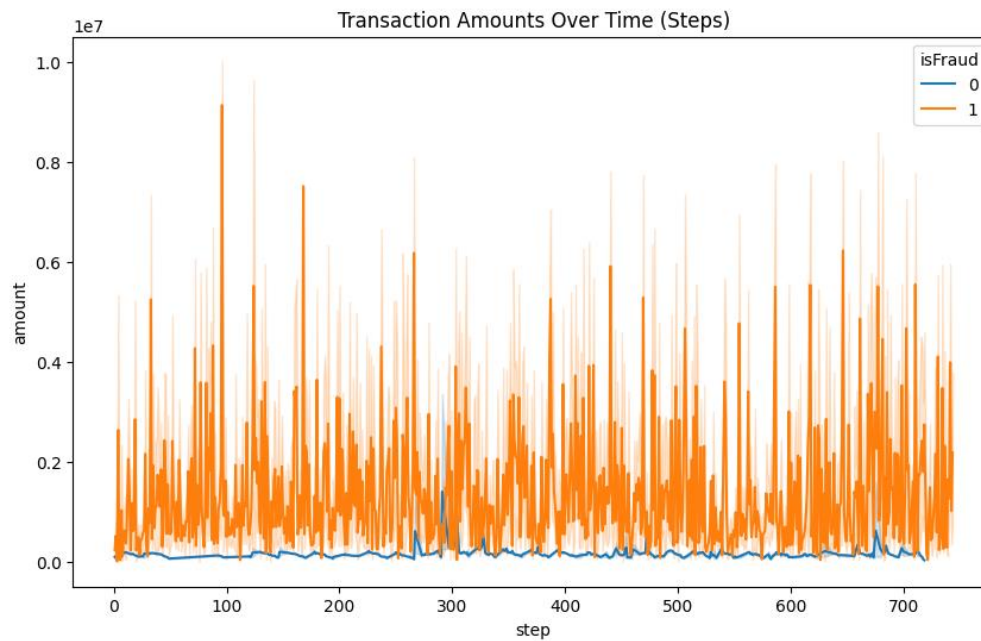
8.6 Visualization for Data Summarization

Several visualizations were generated to summarize the dataset:

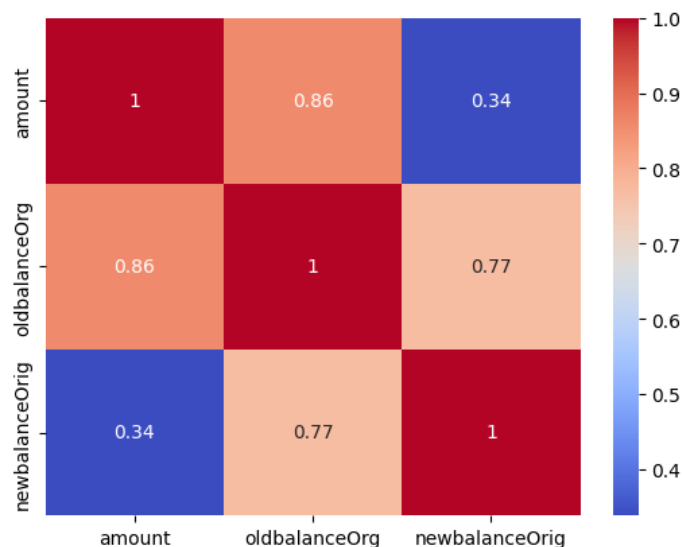
- **Fraudulent vs Non-Fraudulent Transactions by Type:** A count plot was used to compare fraudulent and non-fraudulent transactions across different transaction types.



- **Transaction Amounts Over Time:** A line plot displayed transaction amounts over the time steps, showing clear distinctions between fraud and non-fraud cases.



- **Correlation Heatmap:** A heatmap was generated to visualize correlations between numerical features in fraudulent transactions.



- **Cramér's V:** The association between categorical variables (e.g., transaction type) and fraud was measured, showing a weak correlation between transaction type and fraud.

The pre-processing steps outlined above helped prepare the dataset for fraud detection in financial transactions by creating new features, handling imbalances in the dataset, scaling, and encoding data for machine learning. The synthetic fraud generation ensured the inclusion of all transaction types in the fraud class, thereby enhancing the dataset's utility for model training.

This comprehensive approach to data pre-processing will enable the development of robust fraud detection models, capable of identifying fraudulent activities with greater accuracy.

CHAPTER 9

MACHINE LEARNING MODELLING AND TECHNIQUES APPLIED:

We utilized several approaches, including logistic regression, decision trees, and deep neural networks, to identify fraudulent activities based on provided financial transaction data.

The dataset used for this project contains financial transaction records with class imbalance, where fraudulent transactions are the minority. To address this, Synthetic Minority Over-sampling Technique (SMOTE) was applied. The performance of different models was evaluated based on various metrics such as accuracy, precision, recall, and AUC-ROC.

The dataset consists of 18 features, including transaction amount, balance information, and customer frequency data. Due to the imbalanced nature of the dataset, we applied **SMOTE** to balance the minority fraud class, and then split the data into training and test sets (70/30). The data was scaled using **StandardScaler** for better performance of machine learning algorithms.

The features were engineered to capture important aspects of fraudulent transactions, including balance differences, the ratio of balance decreases, and flags for specific transaction types (transfer, cash-out).

Machine Learning Models Applied

9.1 Logistic Regression Model

Logistic Regression is a simple yet effective model, commonly used for binary classification problems such as fraud detection. The key steps involved were:

- **Class Balancing:** `class_weight='balanced'` to handle class imbalance.
- **Evaluation Metrics:** Accuracy, Confusion Matrix, Classification Report, and AUC-ROC score were used for evaluation.
- **Cross-Validation:** 5-fold cross-validation was applied to assess the model's robustness.

Model Performance:

- Accuracy: 95.17%
- AUC-ROC Score: 0.995
- Cross-Validation AUC-ROC Score: 0.9947

The model showed strong AUC-ROC performance, indicating it is effective in distinguishing between fraudulent and non-fraudulent transactions. However, precision and recall for the fraud class were relatively low due to the imbalanced nature of the dataset.

9.2 CatBoost Classifier

CatBoost is a gradient-boosting algorithm that efficiently handles categorical features and class imbalance. Parameters were tuned using Optuna for better performance.

Model Performance:

- Accuracy: 99.99%
- AUC-ROC Score: 0.9994
- Cross-Validation AUC-ROC Score: 0.9995

The CatBoost model significantly outperformed Logistic Regression in both accuracy and AUC-ROC. The high recall and precision values for the fraud class indicate that CatBoost can detect fraudulent transactions with high confidence.

9.3 Decision Tree Model

A Decision Tree classifier was applied with class balancing to handle the imbalanced dataset. The maximum depth was limited to 10 to avoid overfitting.

Model Performance:

- Accuracy: 99.98%
- AUC-ROC Score: 0.9983
- Cross-Validation AUC-ROC Score: 0.9988

The Decision Tree model performed well but was slightly less accurate than the CatBoost model. It achieved a high recall for the fraud class, making it suitable for scenarios where identifying fraud is the top priority.

9.4 Deep Neural Network (DNN)

A Deep Neural Network with multiple hidden layers was designed to capture complex patterns in the data. Techniques such as batch normalization and dropout were applied to prevent overfitting.

Model Performance:

- Accuracy: 99.94%
- AUC-ROC Score: 0.9952
- Cross-Validation AUC-ROC Score: 0.9947

The DNN showed high accuracy and AUC-ROC scores, proving to be effective in detecting fraud. However, the training time was significantly longer compared to other models, making it less practical in real-time fraud detection.

9.5 Comparative Analysis of Other Models

Other classification models, including Support Vector Machine (SVM), Gradient Boosting, K-Nearest Neighbors (KNN), Naive Bayes, and XGBoost, were also evaluated. The results are summarized below:

- **SVM:** AUC-ROC Score: 0.9916, Accuracy: 99.94%
- **Gradient Boosting:** AUC-ROC Score: 0.7689, Accuracy: 99.91%

- **KNN**: AUC-ROC Score: 0.9998, Accuracy: 99.95%
- **Naive Bayes**: AUC-ROC Score: 0.9919, Accuracy: 93.17%
- **XGBoost**: AUC-ROC Score: 0.9999, Accuracy: 99.99%

Among these models, **XGBoost** showed the highest AUC-ROC score, followed closely by **KNN** and **SVM**.

In this project, several machine learning models were applied to the problem of fraud detection in financial transactions. While simpler models like Logistic Regression and Decision Tree achieved good results, more advanced techniques such as CatBoost, XGBoost, and DNN performed significantly better, with **CatBoost** and **XGBoost** emerging as the most effective models.

These results underscore the importance of model selection and hyperparameter tuning in fraud detection, especially when dealing with imbalanced datasets. Combining models or implementing an ensemble approach may offer further improvements in real-world applications.

CHAPTER 10

CODE AND SCREENSHOTS

Jupyter fraud-detection-in-financial-transactions (2) Last Checkpoint: 2 hours ago

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

Problem Statement:
Develop a machine learning model to detect fraudulent transactions in real-time, enhancing the security of financial systems.

Introduction:
Fraudulent financial transactions can lead to significant losses. Detecting fraud in real-time can enhance the security of financial systems.

Relevance:
Detecting fraud in real-time is essential for financial institutions to protect their customers and assets.

Data Source:
Financial Transactions Dataset.

Load all the required Libraries

```
[1]: # General-purpose Libraries
import os
import gc
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Machine Learning Libraries
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler, OneHotEncoder, LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             classification_report, roc_auc_score,
                             roc_curve)
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans
from sklearn.datasets import make_classification
from scipy.stats import chi2_contingency
```

Fraud Detection in Finan Document1 - Word Document2 - Word fraud-detection-in-fin

ENG IN 19:47:53 09-10-2024

Jupyter fraud-detection-in-financial-transactions (2) Last Checkpoint: 2 hours ago

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

```
# Handling imbalanced datasets
from imblearn.over_sampling import SHOTE

# Deep Learning Libraries
from tensorflow.keras import layers, models
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.callbacks import EarlyStopping

# Hugging Face transformers for pre-trained models and tokenizers
from transformers import AutoModelForSequenceClassification, AutoTokenizer

# CatBoost
from catboost import CatBoostClassifier

# XGBoost
import xgboost as xgb

# PyTorch
import torch

# Utility libraries
import joblib
```

Data load

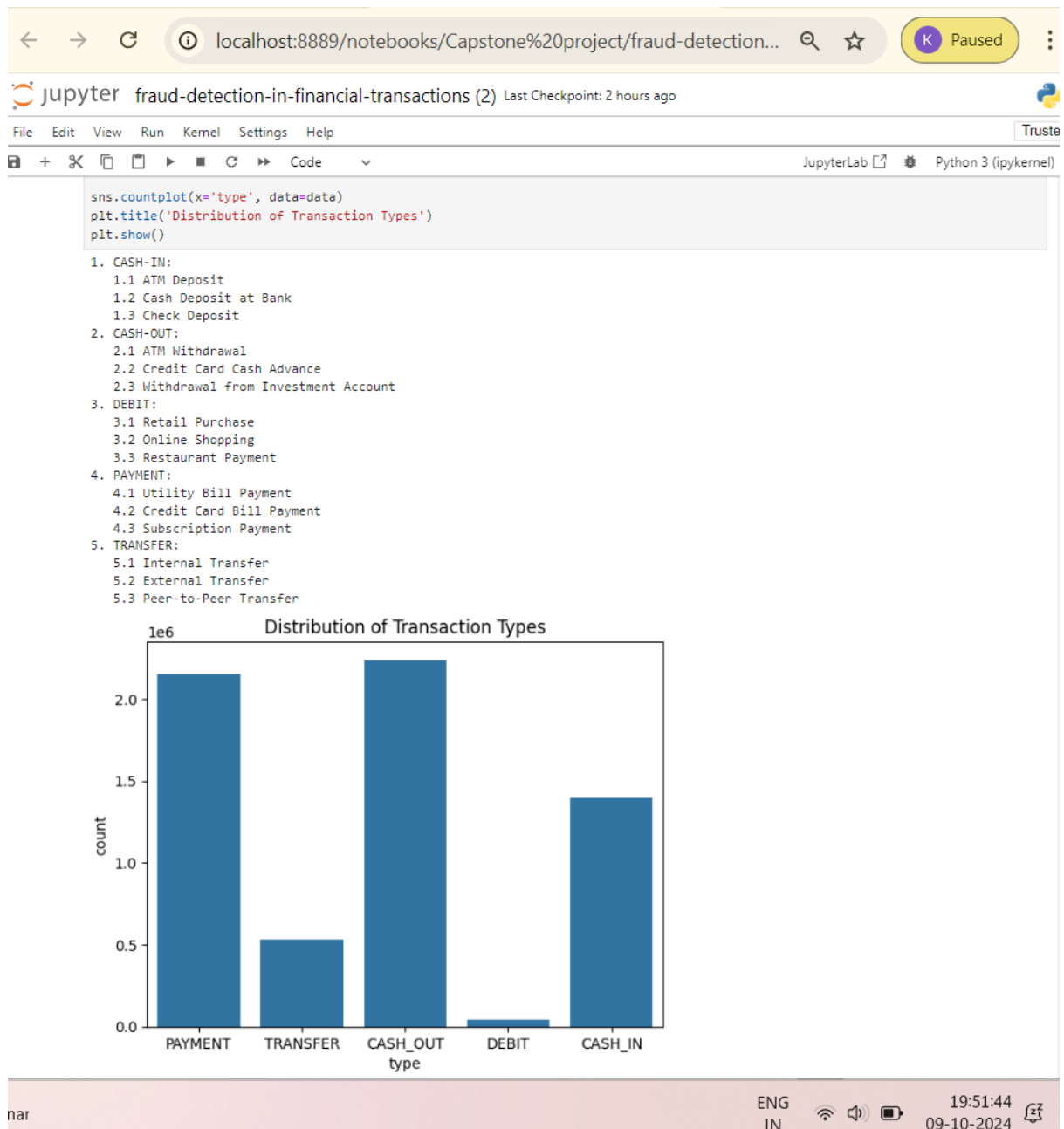
```
[2]: data = pd.read_csv('Synthetic_Financial_datasets_log.csv')
```

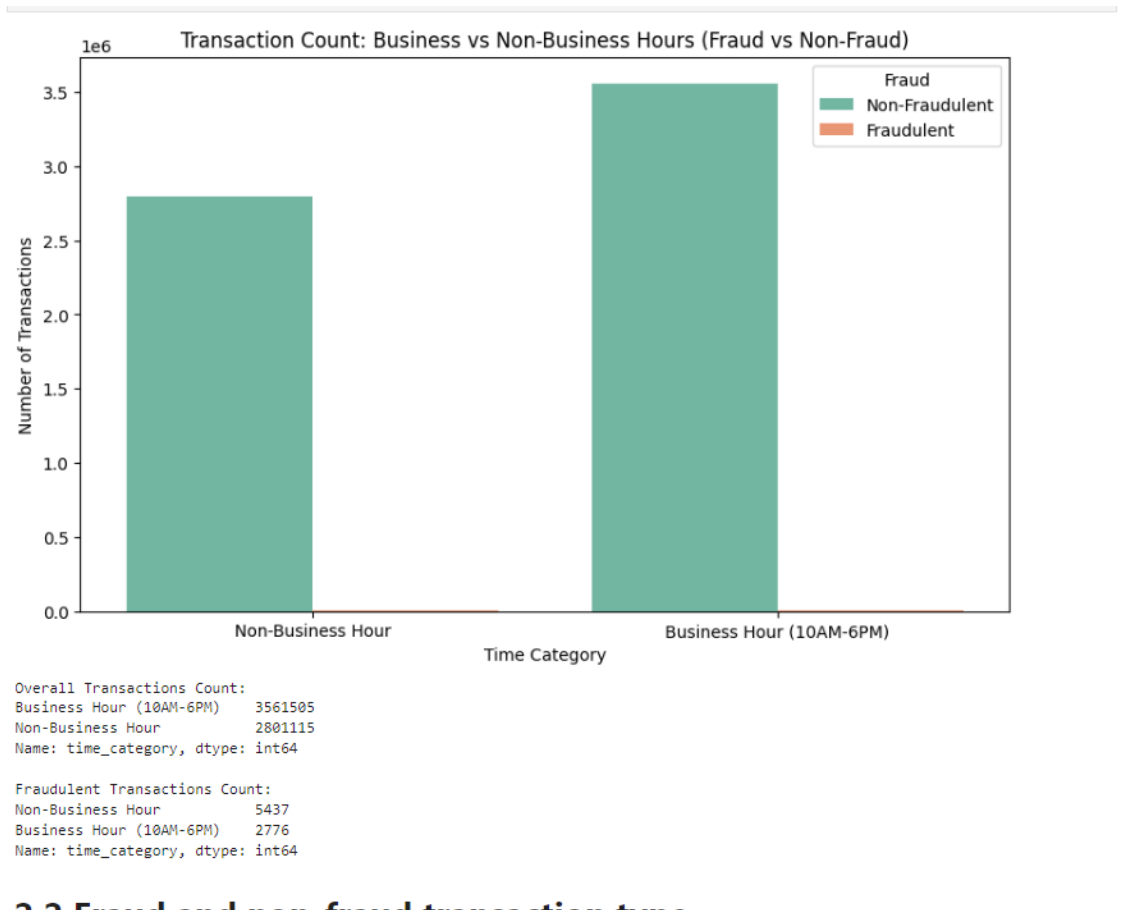
Transaction Dataset Field Summary

```
[3]: fields = {
    "Step": "Represents a unit of time in hours. The simulation spans 744 steps (equivalent to 31 days).",
    "type": "The type of transaction, which includes categories like CASH-IN, CASH-OUT, DEBIT, PAYMENT, and TRANSFER.",
    "amount": "The transaction amount in the local currency.",
    "nameOrig": "The ID of the customer initiating the transaction.",
    "oldbalanceOrig": "The balance of the customer before the transaction.",
    "newbalanceOrig": "The balance of the customer after the transaction.",
    "nameDest": "The ID of the recipient of the transaction.",
    "oldbalanceDest": "The balance of the recipient before the transaction.",
}
```

Document1 - Word Document2 - Word fraud-detection-in-fin

ENG IN 19:49:06 09-10-2024





2.2 Fraud and non-fraud transaction type

ar

ENG
IN



19:53:54
09-10-2024



2.2 Fraud and non-fraud transaction type

```
[11]: fraud_transactions = data[data['isFraud'] == 1]
non_fraud_transactions = data[data['isFraud'] == 0]

print("fraud_transactions", fraud_transactions['type'].value_counts())
print("non_fraud_transactions", non_fraud_transactions['type'].value_counts())
```

```
fraud_transactions CASH_OUT    4116
TRANSFER          4097
Name: type, dtype: int64
non_fraud_transactions CASH_OUT    2233384
PAYMENT          2151495
CASH_IN          1399284
TRANSFER          528812
DEBIT             41432
Name: type, dtype: int64
```

2.3 Count of fraudulent transactions based on Merchant and Customer

```
[12]: fraud_data = data[data['isFraud'] == 1]
count_M = fraud_data[fraud_data['nameDest'].str.startswith('M')].shape[0]
count_C = fraud_data[fraud_data['nameDest'].str.startswith('C')].shape[0]
print(f"Count of fraudulent transactions where nameDest is Merchant: {count_M}")
print(f"Count of fraudulent transactions where nameDest is customer: {count_C}")
```

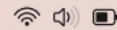
```
Count of fraudulent transactions where nameDest is Merchant: 0
Count of fraudulent transactions where nameDest is customer: 8213
```

```
[13]: #Compare fraud cases across transaction types
sns.countplot(x='type', hue='isFraud', data=data)
plt.title('Fraudulent vs Non-Fraudulent Transactions by Type')
plt.show()
```

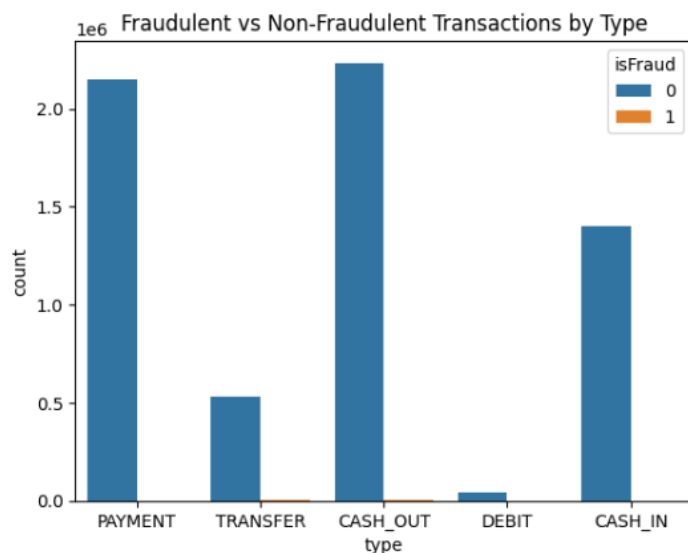
1e6 Fraudulent vs Non-Fraudulent Transactions by Type

nar

ENG
IN



19:55:04
09-10-2024



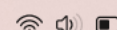
2.4 Plot the number of transactions over time steps

```
[14]: transactions_per_step = data.groupby('step').size()
transactions_per_step.plot(kind='line', title='Transaction Volume Over Time')
#insights: Identifying the peaks in transaction activity. Certain hours with more activity or spikes
```

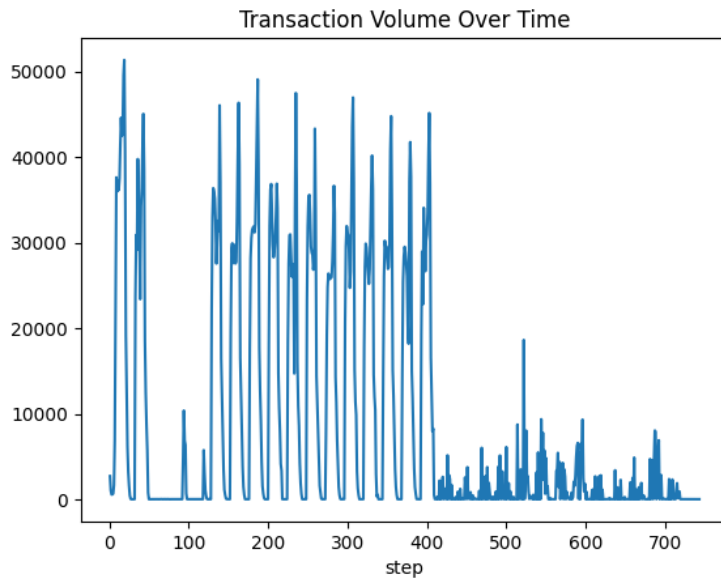
```
[14]: <Axes: title={'center': 'Transaction Volume Over Time'}, xlabel='step'>
```

nar

ENG
IN

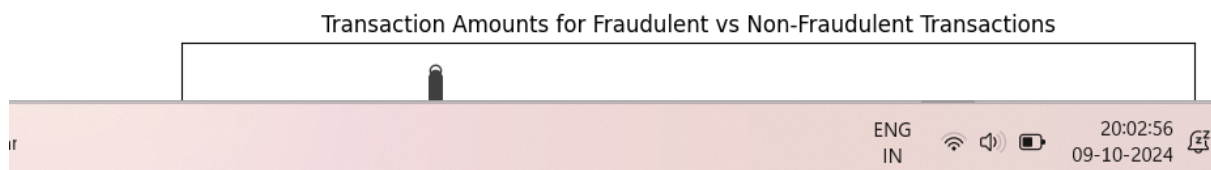


19:55:45
09-10-2024



2.5 Compare transaction amounts for fraud and non-fraud cases ,Use log scale to handle wide range of values

```
[15]: plt.figure(figsize=(10, 6))
sns.boxplot(x='isFraud', y='amount', data=data)
plt.title('Transaction Amounts for Fraudulent vs Non-Fraudulent Transactions')
plt.yscale('log')
plt.show()
```





2.6 Plot old balance origin vs. new balance origin

```
[16]: plt.figure(figsize=(10, 6))
sns.scatterplot(x='oldbalanceOrig', y='newbalanceOrig', hue='isFraud', data=data, alpha=0.5)
plt.title('Old Balance vs New Balance (Origin) for Fraudulent vs Non-Fraudulent Transactions')
plt.xscale('log')
plt.yscale('log')
plt.show()
```

lar

Spinning Tool

ENG
IN

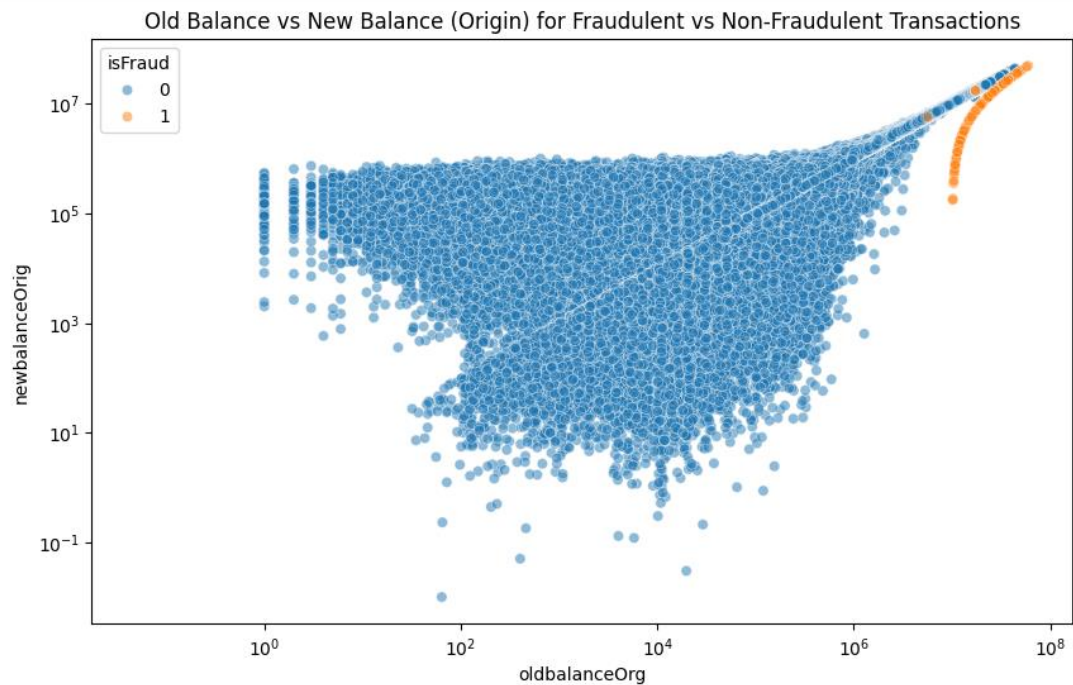


20:05:31
09-10-2024



2.6 Plot old balance origin vs. new balance origin

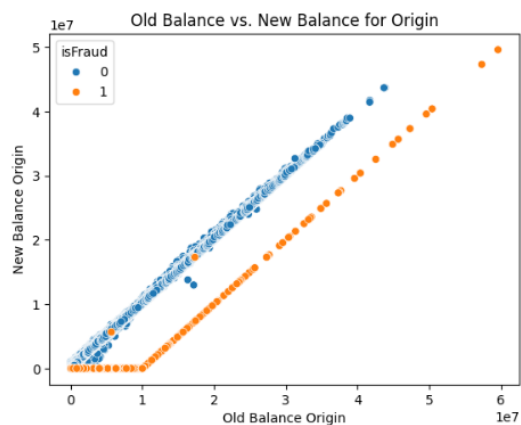
```
[16]: plt.figure(figsize=(10, 6))
sns.scatterplot(x='oldbalanceOrig', y='newbalanceOrig', hue='isFraud', data=data, alpha=0.5)
plt.title('Old Balance vs New Balance (Origin) for Fraudulent vs Non-Fraudulent Transactions')
plt.xscale('log')
plt.yscale('log')
plt.show()
```

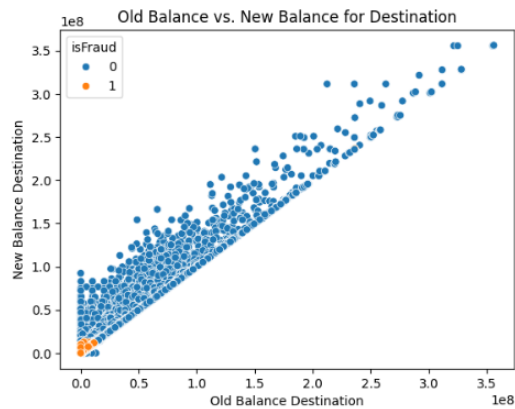


2.7 Scatter plot of old balance vs. new balance for origin.

```
[17]: ##This scatter plot examines the relationship between the old balance (oldbalanceOrg) and the new balance (newbalanceOrig) for the origin account, with
sns.scatterplot(x='oldbalanceOrg', y='newbalanceOrig', hue='isFraud', data=data)
plt.title('Old Balance vs. New Balance for Origin')
plt.xlabel('Old Balance Origin')
plt.ylabel('New Balance Origin')
plt.show()
##Relationship Insight: Helps identify if there's a particular pattern in the old vs. new balance related to fraud.
##Fraud Detection: You can see if fraudulent transactions have distinctive patterns in balance changes.
##If fraudulent transactions show specific patterns (larger changes in balance), this could help in designing better fraud detection mechanisms.

# Scatter plot of old balance vs. new balance for destination
##Similar to the previous scatter plot, this one explores the relationship between the old balance (oldbalanceDest) and new balance (newbalanceDest), but
sns.scatterplot(x='oldbalanceDest', y='newbalanceDest', hue='isFraud', data=data)
plt.title('Old Balance vs. New Balance for Destination')
plt.xlabel('Old Balance Destination')
plt.ylabel('New Balance Destination')
plt.show()
##Relationship Insight: Helps to understand how the transaction affects the destination balance and whether fraud is associated with specific balance changes.
##Fraud Detection: Identifies if there are any noticeable patterns in balance changes for fraudulent transactions.
##Help to find that fraudulent transactions to certain types of destination accounts have specific balance change characteristics, which could be indicative
```





2.8 Percentage of Fraud by Transaction Type

```
[18]: # Filter the DataFrame for fraudulent and non-fraudulent transactions
fraud_data = data[data['isFraud'] == 1]
non_fraud_data = data[data['isFraud'] == 0]
# Calculate the counts of different transaction types in fraudulent transactions
transaction_type_fraud_count = fraud_data['type'].value_counts()
# Calculate the percentage of fraud for each transaction type
percentage = (transaction_type_fraud_count / transaction_type_fraud_count.sum()) * 100
# Print the percentage of fraud for each transaction type
print(percentage)

# Plot the percentage of fraud for each transaction type
plt.figure(figsize=(8, 6))
percentage.plot(kind='bar')

plt.xlabel('Transaction Type')
plt.ylabel('Percentage')
plt.title('Percentage of Fraud by Transaction Type')
plt.xticks(rotation=45)
plt.show()

### So only 'cash out' and 'Transfer' have fraudulent transaction. org has to focus in these two type.

#Potential Issues:
```

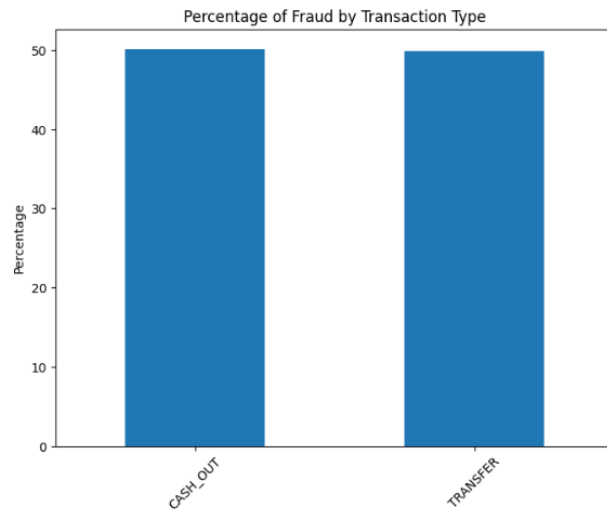
```
#How to address the problem
#Collect More Data:
##Expand Dataset: If possible, gather more data, especially for transaction types that currently have no fraudulent transactions. This can help ensure t
##Synthetic Data Generation: Use techniques like SMOTE (Synthetic Minority Over-sampling Technique) to generate synthetic examples for transaction types

#Resampling Techniques:
##Oversampling: Increase the number of instances of fraud in transaction types that currently have none by replicating or synthetically generating more
##Undersampling: Decrease the number of non-fraudulent samples in transaction types with many instances to balance the dataset.

#Feature Engineering:
##Include Transaction Type as a Feature: Incorporate the transaction type as a feature in the model, so the model learns to associate fraud with transac
##Interaction Features: Create interaction features that combine transaction type with other features to better capture patterns specific to each type.

#Model Evaluation:
#Cross-Validation: Use techniques like k-fold cross-validation to ensure that the model's performance is evaluated on different subsets of the data, hel
#Class Weights: Adjust class weights in your model to give more importance to less frequent classes, which can help the model focus more on the underrep
```

```
CASH_OUT    50.11567
TRANSFER    49.88433
Name: type, dtype: float64
```



2.9 Label encoding

```
[19]: # One-Hot Encode the 'type' column to include it in correlation
data_encoded = pd.get_dummies(data, columns=['type'], drop_first=False)
```

2.10 Handling missing Fraudulent transaction for Debit/Cash-in/Payment

1. Use the Existing Dataset to Understand Correlations

First, analyze the relationships between variables (e.g., amount, oldbalanceOrig, newbalanceOrig) across transaction types with observed fraud (CASH_OUT and TRANSFER). You can use statistical analysis, correlation matrices, or more sophisticated techniques like clustering

Method-1 - K-means Clustering

```
[20]: objtlist = data.select_dtypes(include = "object").columns
label_encoder = LabelEncoder()

for i in objtlist:
    data[i] = label_encoder.fit_transform(data[i].astype(str))

correlation = data.corr()
print(correlation["isFraud"].sort_values(ascending=False))

# Correlation matrix for CASH_OUT and TRANSFER transactions
fraud_data = data[data["isFraud"] == 1]
corr_matrix = fraud_data[["amount", "oldbalanceOrig", "newbalanceOrig"]].corr()

# Plot correlation heatmap
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
plt.show()
```

isFraud	1.000000
amount	0.076688
isFlaggedFraud	0.044109
step	0.031578
type	0.020833
oldbalanceOrig	0.010154
newbalanceDest	0.000535
nameOrig	-0.000464
oldbalanceDest	-0.005985
newbalanceOrig	-0.008148
nameDest	-0.021963

Name: isFraud, dtype: float64



Loop: The process is repeated for each transaction type (DEBIT, CASH_IN, PAYMENT) that doesn't have any fraudulent transactions.

Correlation-based: Synthetic data is generated using the learned patterns from existing fraud cases, avoiding random data generation.

Cluster-based: This ensures that the synthetic fraud data resembles the actual fraud behavior seen in CASH_OUT and TRANSFER.

```
[22]: # Assuming 'non_fraud_data' is the dataset containing DEBIT, CASH_IN, and PAYMENT without fraud
# and 'fraud_data' contains CASH_OUT and TRANSFER with fraud

# Step 1: Correlation analysis on CASH_OUT and TRANSFER
fraud_only = fraud_data[fraud_data['isFraud'] == 1]
features = ['amount', 'oldbalanceorg', 'newbalanceOrig'] # Add more numerical features if needed

# Perform clustering on fraud cases
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
fraud_only['cluster'] = kmeans.fit_predict(fraud_only[features])

# Analyze the feature distribution for each cluster
fraud_clusters = fraud_only.groupby('cluster').mean(numeric_only=True)

# Step 2: Use correlation from fraud clusters to generate synthetic fraud data
def generate_synthetic_data(non_fraud_subset, fraud_clusters, categorical_columns, n_samples=100):
    """Generate synthetic fraud data based on non-fraud and cluster distributions."""
    synthetic_data = []
    for _, cluster in fraud_clusters.iterrows():
        for _ in range(n_samples):
            # Randomly sample from the non-fraud data to create a fraud-like example
            sample = non_fraud_subset.sample(1).copy()

            # Adjust sample features based on cluster mean
            for feature in features:
                sample[feature] = np.random.normal(loc=cluster[feature], scale=0.1*cluster[feature])

            sample['isFraud'] = 1 # Label as fraud

            # Keep categorical columns unchanged
            for cat_col in categorical_columns:
                sample[cat_col] = non_fraud_subset[cat_col].iloc[0]

            synthetic_data.append(sample)

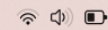
    return pd.concat(synthetic_data)

# Categorical columns you want to preserve
categorical_columns = ['nameOrig', 'nameDest', 'type'] # Add other categorical columns if needed

# Step 3: Apply the synthetic generation for each transaction type without fraud
synthetic_fraud_data = []
```

fraud-detection-in-finar

ENG
IN



20:09:55
09-10-2024

```
for transaction_type in ['DEBIT', 'CASH_IN', 'PAYMENT']:
    print(f"Generating synthetic fraud data for {transaction_type}")

    # Filter non-fraud cases for the specific transaction type
    non_fraud_subset = non_fraud_data[non_fraud_data['type'] == transaction_type]

    if non_fraud_subset.empty:
        print(f"No non-fraud data for {transaction_type}, skipping...")
        continue

    # Generate synthetic fraud data for the type based on fraud cluster correlations
    synthetic_data = generate_synthetic_data(non_fraud_subset, fraud_clusters, categorical_columns, n_samples=500)

    # Add the type column back (though it should be retained via categorical columns)
    synthetic_data['type'] = transaction_type

    synthetic_fraud_data.append(synthetic_data)

# Combine the synthetic fraud data
if synthetic_fraud_data:
    synthetic_fraud_data_combined = pd.concat(synthetic_fraud_data, ignore_index=True)

    # Add synthetic fraud data to the original dataset
    complete_dataset = pd.concat([non_fraud_data, fraud_data, synthetic_fraud_data_combined], ignore_index=True)

    print("Synthetic fraud data generation complete!")
else:
    print("No synthetic fraud data was generated.")
```

```
Generating synthetic fraud data for DEBIT
Generating synthetic fraud data for CASH_IN
Generating synthetic fraud data for PAYMENT
Synthetic fraud data generation complete!
```

fraud-detection-in-finar

ENG
IN



20:10:23
09-10-2024

Method-2 SMOTE

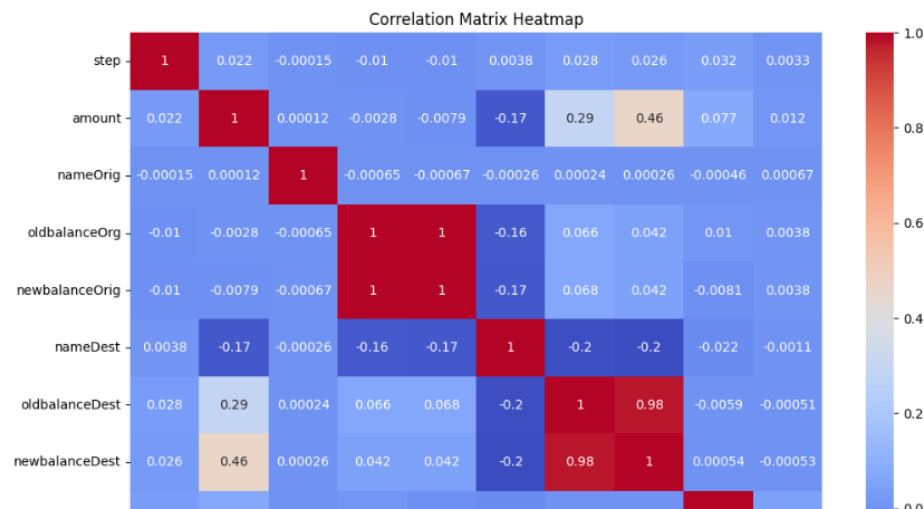
```
[23]: #Identify the correlation among fields and types
# One-Hot Encode the 'type' column to include it in correlation
data_encoded = pd.get_dummies(data, columns=['type'], drop_first=False)

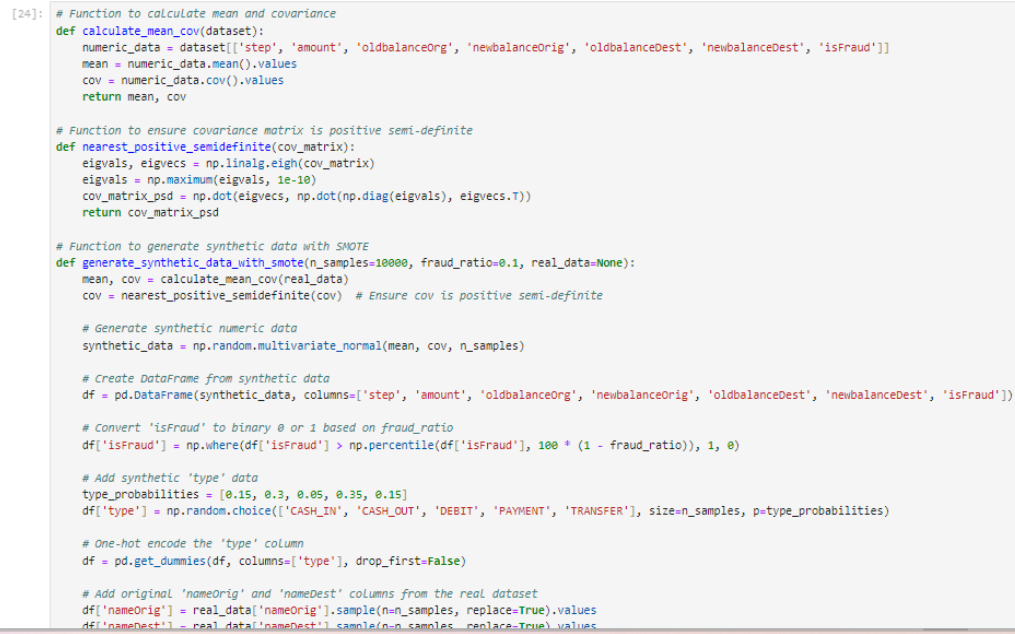
# Apply Label Encoding for 'nameOrig' and 'nameDest' columns
label_encoder = LabelEncoder()
data_encoded['nameOrig'] = label_encoder.fit_transform(data_encoded['nameOrig'])
data_encoded['nameDest'] = label_encoder.fit_transform(data_encoded['nameDest'])

# Select only numeric columns, which now include the one-hot encoded 'type' columns and Label-encoded 'nameOrig', 'nameDest'
numeric_columns = data_encoded.select_dtypes(include=[float, int, bool])

# Generate the correlation matrix
correlation_matrix = numeric_columns.corr()

# Plot the heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix Heatmap')
plt.show()
```





```

# Add original 'nameOrig' and 'nameDest' columns from the real dataset
df['nameOrig'] = real_data['nameOrig'].sample(n=n_samples, replace=True).values
df['nameDest'] = real_data['nameDest'].sample(n=n_samples, replace=True).values

# Exclude 'nameOrig' and 'nameDest' for scaling
X = df.drop(['isFraud', 'nameOrig', 'nameDest'], axis=1)
y = df['isFraud']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply SMOTE to balance classes
smote = SMOTE(sampling_strategy='auto', random_state=42)
X_resampled, y_resampled = smote.fit_resample(X_scaled, y)

# Rebuild the DataFrame after SMOTE
df_resampled = pd.DataFrame(X_resampled, columns=X.columns)
df_resampled['isFraud'] = y_resampled

# Add 'nameOrig' and 'nameDest' back to the resampled DataFrame
df_resampled['nameOrig'] = df['nameOrig'].sample(n=len(df_resampled), replace=True).values
df_resampled['nameDest'] = df['nameDest'].sample(n=len(df_resampled), replace=True).values

return df_resampled

# Generate synthetic data with SMOTE
synthetic_data_smote = generate_synthetic_data_with_smote(n_samples=20000, fraud_ratio=0.2, real_data=data)

# View the distribution
print(synthetic_data_smote['isFraud'].value_counts())

# Save the synthetic data
synthetic_data_smote.to_csv('synthetic_fraud_detection_data_smote.csv', index=False)

```

```

0    16000
1    16000
Name: isFraud, dtype: int64

```

```

# Calculate the counts of fraudulent transactions
# Ensure the values in type columns are properly binary (0 or 1)
# First, we'll inspect the unique values in these columns
for col in ['type_CASH_IN', 'type_CASH_OUT', 'type_DEBIT', 'type_PAYMENT', 'type_TRANSFER']:
    print(f"Unique values in {col} before fixing:", synthetic_data_smote[col].unique())

```

```

# Clip the values in the type columns to ensure they are between 0 and 1

```

fraud-detection-in-finar

ENG
IN



20:13:09
09-10-2024

```

print(f"Unique values in {col} before fixing:", synthetic_data_smote[col].unique())

# Clip the values in the type columns to ensure they are between 0 and 1
synthetic_data_smote[['type_CASH_IN', 'type_CASH_OUT', 'type_DEBIT', 'type_PAYMENT', 'type_TRANSFER']] = \
    synthetic_data_smote[['type_CASH_IN', 'type_CASH_OUT', 'type_DEBIT', 'type_PAYMENT', 'type_TRANSFER']].apply(lambda x: x.clip(lower=0, upper=1))

# Verify that the type columns now contain only 0 and 1
for col in ['type_CASH_IN', 'type_CASH_OUT', 'type_DEBIT', 'type_PAYMENT', 'type_TRANSFER']:
    print(f"Unique values in {col} after fixing:", synthetic_data_smote[col].unique())

# Filter for fraudulent transactions
fraudulent_transactions = synthetic_data_smote[synthetic_data_smote['isFraud'] == 1]

# Calculate the counts of fraudulent transactions by summing the one-hot encoded type columns
fraud_counts_by_type = pd.DataFrame({
    'CASH_IN': fraudulent_transactions['type_CASH_IN'].sum(),
    'CASH_OUT': fraudulent_transactions['type_CASH_OUT'].sum(),
    'DEBIT': fraudulent_transactions['type_DEBIT'].sum(),
    'PAYMENT': fraudulent_transactions['type_PAYMENT'].sum(),
    'TRANSFER': fraudulent_transactions['type_TRANSFER'].sum()
}, index=['Fraudulent Transactions']).T

# Output the corrected summary table
print("\nFraudulent Transactions by Type:")
print(fraud_counts_by_type)

# Total number of fraudulent transactions
total_fraudulent_transactions = fraudulent_transactions['isFraud'].count()
print(f"\nTotal Fraudulent Transactions: {total_fraudulent_transactions}")

Unique values in type_CASH_IN before fixing: [-0.42616204  2.34652528]
Unique values in type_CASH_OUT before fixing: [-0.65847475  1.51866113]
Unique values in type_DEBIT before fixing: [-0.22985712  4.35431738]
Unique values in type_PAYMENT before fixing: [ 1.38001587 -0.72462935]
Unique values in type_TRANSFER before fixing: [-0.41917757  2.38562383]
Unique values in type_CASH_IN after fixing: [0. 1.]
Unique values in type_CASH_OUT after fixing: [0. 1.]
Unique values in type_DEBIT after fixing: [0. 1.]
Unique values in type_PAYMENT after fixing: [1. 0.]
Unique values in type_TRANSFER after fixing: [0. 1.]

Fraudulent Transactions by Type:
Fraudulent Transactions
CASH_IN      2337.0
CASH_OUT     4984.0
DEBIT         812.0
PAYMENT      5473.0
TRANSFER     2394.0

```

 fraud-detection-in-finar

ENG
IN



20:13:40
09-10-2024

2.11 Changing categorical NameOrigin and NameDest to Int using Frequency Encoding as Label encoding for unique Ids is not a good approach

```
[27]: complete_dataset.head()
```

	step	type	amount	nameOrig	oldbalanceOrig	newbalanceOrig	nameDest	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
0	1	PAYMENT	9839.64	C1231006815	170136.0	160296.36	M1979787155	0.0	0.0	0	0
1	1	PAYMENT	1864.28	C1666544295	21249.0	19384.72	M2044282225	0.0	0.0	0	0
2	1	PAYMENT	11668.14	C2048537720	41554.0	29885.86	M1230701703	0.0	0.0	0	0
3	1	PAYMENT	7817.71	C90045638	53860.0	46042.29	M573487274	0.0	0.0	0	0
4	1	PAYMENT	7107.77	C154988899	183195.0	176087.23	M408069119	0.0	0.0	0	0

```
[28]: # Frequency encoding for 'nameOrig' and 'nameDest'
complete_dataset['nameOrig_freq'] = complete_dataset['nameOrig'].map(complete_dataset['nameOrig'].value_counts())
complete_dataset['nameDest_freq'] = complete_dataset['nameDest'].map(complete_dataset['nameDest'].value_counts())

# Drop the original 'nameOrig' and 'nameDest' columns
complete_dataset.drop(['nameOrig', 'nameDest'], axis=1, inplace=True)
```

2.12 Calculating Balance Change Features

```
[29]: #You want to track the decrease in the balance after a transaction and its relation to fraud. You can create new features to capture these insights:
#Balance decrease ratio: Measure how much the balance has decreased after the transaction.
#balance_decrease_ratio=amountoldbalanceOrig-newbalanceOrig
#This shows how much of the original balance was spent.
#Zero balance flag: A feature that flags if the oldbalanceOrig is zero, which could indicate suspicious activity for certain transaction types.

complete_dataset['balance_decrease_ratio'] = (complete_dataset['oldbalanceOrig'] - complete_dataset['newbalanceOrig']) / complete_dataset['amount']
complete_dataset['is_old_balance_zero'] = complete_dataset['oldbalanceOrig'] == 0
complete_dataset['balance_decrease_ratio'].replace([np.inf, -np.inf], np.nan)
# Set NaN values in 'amount_to_oldbalance_ratio' to zero
complete_dataset['balance_decrease_ratio'] = complete_dataset['balance_decrease_ratio'].fillna(0)

#TRANSFER and CASH_OUT transactions might be more likely to involve fraud
complete_dataset['is_type_transfer'] = complete_dataset['type'] == 'TRANSFER'
complete_dataset['is_type_cash_out'] = complete_dataset['type'] == 'CASH_OUT'
complete_dataset.head()
```



fraud-detection-in-finar

ENG
IN



20:14:01
09-10-2024

[29]:	step	type	amount	oldbalanceOrig	newbalanceOrig	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud	nameOrig_freq	nameDest_freq	balance_decrea
0	1	PAYMENT	9839.64	170136.0	160296.36	0.0	0.0	0	0	1501	1501	
1	1	PAYMENT	1864.28	21249.0	19384.72	0.0	0.0	0	0	1	1	
2	1	PAYMENT	11668.14	41554.0	29885.86	0.0	0.0	0	0	1	1	
3	1	PAYMENT	7817.71	53860.0	46042.29	0.0	0.0	0	0	1	1	
4	1	PAYMENT	7107.77	183195.0	176087.23	0.0	0.0	0	0	1	1	

2.13 Scaling/Normalization & One-Hot Encoding

```
[30]: # List of columns to scale/normalize
columns_to_scale = ['step', 'amount', 'oldbalanceOrig', 'newbalanceOrig', 'oldbalanceDest', 'newbalanceDest', 'nameOrig_freq', 'nameDest_freq']

if 'amount_to_oldbalance_ratio' in complete_dataset.columns:
    complete_dataset.drop('amount_to_oldbalance_ratio', axis=1, inplace=True)

# Apply Standard scaling only to numerical columns
scaler = StandardScaler()
complete_dataset[columns_to_scale] = scaler.fit_transform(complete_dataset[columns_to_scale])

# Ensure that after scaling, all columns (both scaled and categorical) are included in the dataset
df_scaled = complete_dataset.copy()

# Apply One-Hot Encoding on the 'type' column
df_scaled = pd.get_dummies(df_scaled, columns=['type'], drop_first=True)

# Retain labels ('isFraud', 'isFlaggedFraud') as they are (not scaled/normalized)
X = df_scaled.drop(['isFraud', 'isFlaggedFraud'], axis=1) # Features
y = df_scaled['isFraud'] # Target variable

# Optional: Check the dataset after scaling and encoding
print(df_scaled.head())
df_scaled.to_csv('Test.csv', index=False)
```

	step	amount	oldbalanceOrig	newbalanceOrig	oldbalanceDest	
0	-1.703017	-0.275371	-0.229808	-0.237855	-0.323785	\
1	-1.703017	-0.287978	-0.280684	-0.285775	-0.323785	
2	-1.703017	-0.272480	-0.273745	-0.282204	-0.323785	
3	-1.703017	-0.278567	-0.269540	-0.276710	-0.323785	
4	-1.703017	-0.279689	-0.225346	-0.232485	-0.323785	

3. Model

```
[31]: # Split the dataset into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Convert to DataFrames for saving
X_train_scaled_df = pd.DataFrame(X_train_scaled, columns=X_train.columns)
X_test_scaled_df = pd.DataFrame(X_test_scaled, columns=X_test.columns)
y_train_df = pd.DataFrame(y_train.values, columns=['isFraud'])
y_test_df = pd.DataFrame(y_test.values, columns=['isFraud'])

os.makedirs('train_test_data', exist_ok=True)

X_train_scaled_df.to_csv('X_train_scaled.csv', index=False)
X_test_scaled_df.to_csv('X_test_scaled.csv', index=False)
y_train_df.to_csv('y_train.csv', index=False)
y_test_df.to_csv('y_test.csv', index=False)

print("Training and testing data saved in train_test_data.")

Training and testing data saved in train_test_data.
```

3.1 Logistic Regression Model

```
[32]: # Initialize Logistic Regression with class balancing
log_reg = LogisticRegression(class_weight='balanced', max_iter=500)
log_reg.fit(X_train_scaled, y_train)

# Make predictions
y_pred_log_reg = log_reg.predict(X_test_scaled)

# Evaluate Logistic Regression
print("Logistic Regression:")
print("Accuracy:", accuracy_score(y_test, y_pred_log_reg))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_log_reg))
print("Classification Report:\n", classification_report(y_test, y_pred_log_reg))

# AUC-ROC Score
# Get the probabilities for the positive class (fraud)
y_proba_log_reg = log_reg.predict_proba(X_test_scaled)[:, 1]
```

on-in-finar

ENG
IN



20:15:01
09-10-2024




```

# AUC-ROC Score
# Get the probabilities for the positive class (fraud)
y_proba_log_reg = log_reg.predict_proba(X_test_scaled)[: , 1]
roc_score = roc_auc_score(y_test, y_proba_log_reg)
print(f"AUC-ROC Score: {roc_score}")

# Plot ROC curve
fpr, tpr, thresholds = roc_curve(y_test, y_proba_log_reg)
plt.figure(figsize=(8,6))
plt.plot(fpr, tpr, color='blue', label=f"ROC Curve (AUC = {roc_score:.2f})")
plt.plot([0, 1], [0, 1], color='grey', linestyle='--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Receiver Operating Characteristic (ROC) Curve")
plt.legend()
plt.show()

# Cross-Validation
cv_scores = cross_val_score(log_reg, X_train_scaled, y_train, cv=5, scoring='roc_auc')
print(f"Cross-Validation AUC-ROC Scores: {cv_scores}")
print(f"Mean Cross-Validation AUC-ROC Score: {cv_scores.mean()}")

# Save the trained model to a file
model_filename = 'Logistic_Regression_model.pkl'
joblib.dump(log_reg, model_filename)

```

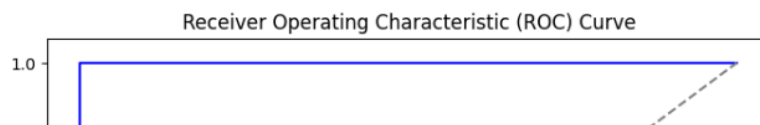
```

Logistic Regression:
Accuracy: 0.9999989529541352
Confusion Matrix:
[[1906318      2]
 [      0  3816]]
Classification Report:

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1906320
1	1.00	1.00	1.00	3816
accuracy			1.00	1910136
macro avg	1.00	1.00	1.00	1910136
weighted avg	1.00	1.00	1.00	1910136

AUC-ROC Score: 0.999998397144468



n-finarr

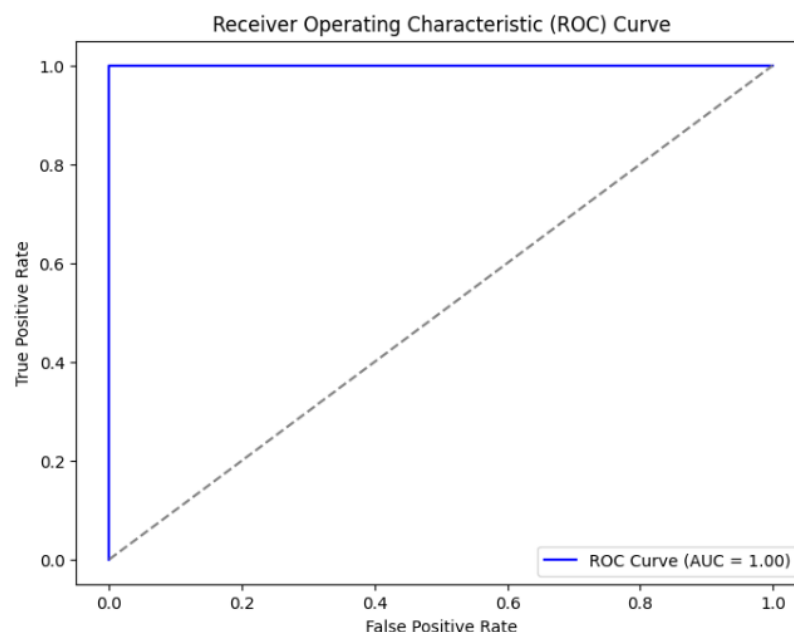
ENG
IN



20:15:23
09-10-2024



AUC-ROC Score: 0.999998397144468



```

Cross-Validation AUC-ROC Scores: [0.99999951 1. 0.99999961 1. 1. ]
Mean Cross-Validation AUC-ROC Score: 0.999998237549279

```

```
[32]: ['Logistic_Regression_model.pkl']
```

n-in-finarr

ENG
IN



20:16:26
09-10-2024



3.2 CatBoostClassifier

```
[33]: # Initialize CatBoostClassifier with class balancing
catboost_model = CatBoostClassifier(class_weights=[1, 10], iterations=500, verbose=False)

catboost_params_optuna = {
    'depth': 2,
    'l2_leaf_reg': 0.0687356140243106,
    'learning_rate': 0.0888501063398756,
    'colsample_bylevel': 0.7017148360901887,
    'subsample': 0.814719415526711,
    'random_seed': 42,
    'eval_metric': 'AUC',
    'verbose': False,
    'loss_function': 'Logloss',
}

# catboost model
catboost_model = CatBoostClassifier(**catboost_params_optuna)
# Train the CatBoost model
catboost_model.fit(X_train_scaled, y_train)

# Make predictions
y_pred_catboost = catboost_model.predict(X_test_scaled)

# Evaluate CatBoost
print("CatBoost Classifier:")
print("Accuracy:", accuracy_score(y_test, y_pred_catboost))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_catboost))
print("Classification Report:\n", classification_report(y_test, y_pred_catboost))

# AUC-ROC Score
# Get the probabilities for the positive class (fraud)
y_proba_catboost = catboost_model.predict_proba(X_test_scaled)[:, 1]
roc_score_catboost = roc_auc_score(y_test, y_proba_catboost)
print(f"AUC-ROC Score: {roc_score_catboost}")

# Plot ROC curve
fpr_catboost, tpr_catboost, thresholds_catboost = roc_curve(y_test, y_proba_catboost)
plt.figure(figsize=(8, 6))
plt.plot(fpr_catboost, tpr_catboost, color='green', label=f"ROC Curve (AUC = {roc_score_catboost:.2f})")
plt.plot([0, 1], [0, 1], color='grey', linestyle='--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Receiver Operating Characteristic (ROC) Curve - CatBoost")
```

fraud-detection-in-finar

20:17:53 09-10-2024

Jupyter fraud-detection-in-financial-transactions (2) Last Checkpoint: 3 hours ago

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

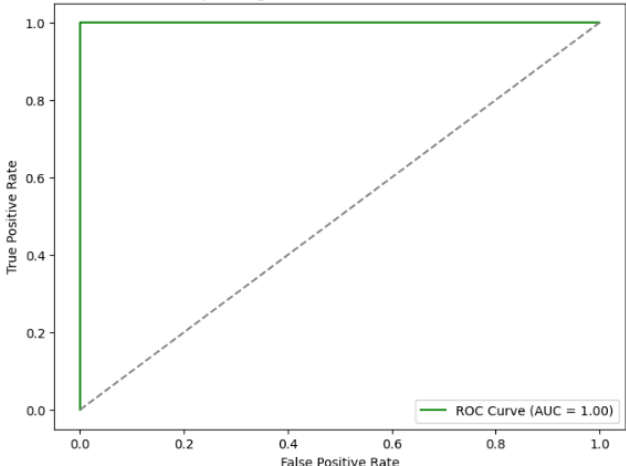
```
cv_scores_catboost = cross_val_score(catboost_model, X_train_scaled, y_train, cv=5, scoring='roc_auc')
print(f"Cross-Validation AUC-ROC Scores: {cv_scores_catboost}")
print(f"Mean Cross-Validation AUC-ROC Score: {cv_scores_catboost.mean()}")

# Save the trained CatBoost model to a file
catboost_model_filename = 'CatBoost_Model.pkl'
joblib.dump(catboost_model, catboost_model_filename)
```

CatBoost Classifier:
Accuracy: 0.9999994764770676
Confusion Matrix:
[[1906320
 [1 3815]]
Classification Report:
precision recall f1-score support
0 1.00 1.00 1.00 1906320
1 1.00 1.00 1.00 3815
accuracy 1.00 1.00 1.00 1910136
macro avg 1.00 1.00 1.00 1910136
weighted avg 1.00 1.00 1.00 1910136

AUC-ROC Score: 1.0

Receiver Operating Characteristic (ROC) Curve - CatBoost



Cross-Validation AUC-ROC Scores: [1. 1. 1. 1. 1.]
Mean Cross-Validation AUC-ROC Score: 0.999999999242052

fraud-detection-in-finar

20:18:28 09-10-2024

3.4 Deep Neural Network Architecture

```
[35]: # Function to build the model
def build_fraud_detection_model(input_shape):
    model = models.Sequential()

    # First Layer: Input Layer
    model.add(layers.Input(shape=input_shape)) # Define input shape using Input Layer

    model.add(layers.Dense(512, activation='relu'))
    model.add(layers.BatchNormalization())
    model.add(layers.Dropout(0.3))

    model.add(layers.Dense(256, activation='relu'))
    model.add(layers.BatchNormalization())
    model.add(layers.Dropout(0.3))

    model.add(layers.Dense(128, activation='relu'))
    model.add(layers.BatchNormalization())

    model.add(layers.Dense(1, activation='sigmoid')) # Output Layer
    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
    return model

# Number of random splits for validation
n_splits = 5
cv_scores = []

# Train-Validate Split
input_shape = (X_train_scaled.shape[1],) # Input shape of the features

for i in range(n_splits):
    print(f"Starting fold {i+1}/{n_splits}")

    X_train_fold, X_val_fold, y_train_fold, y_val_fold = train_test_split(
        X_train_scaled, y_train, test_size=0.2, stratify=y_train, random_state=i)

    model = build_fraud_detection_model(input_shape) # New model instance for each split
    print(f"Training on fold {i+1}...")

    history = model.fit(X_train_fold, y_train_fold, epochs=1, batch_size=64, validation_data=(X_val_fold, y_val_fold), verbose=0)

    # Predict probabilities for the AUC-ROC
    y_val_pred_proba = model.predict(X_val_fold)

    # Calculate ROC AUC score
    auc_score = roc_auc_score(y_val_fold, y_val_pred_proba)
    cv_scores.append(auc_score)

    # Print the results for this fold
    print(f"Fold {i+1} AUC-ROC Score: {auc_score:.4f}")
```

fraud-detection-in-finar

ENG
IN



20:19:38
09-10-2024

```
# Print the results for this fold
print(f"Fold {i+1} AUC-ROC Score: {auc_score:.4f}")
print(f"Fold {i+1} Training Loss: {history.history['loss'][-1]:.4f}, Validation Loss: {history.history['val_loss'][-1]:.4f}")

# Final cross-validation AUC scores
print(f"Cross-Validation AUC Scores: {cv_scores}")
print(f"Mean Cross-Validation AUC Score: {np.mean(cv_scores):.4f}")

# Final evaluation on the test set
print("\nTraining the final model on the full dataset...")
model_final = build_fraud_detection_model(input_shape)
history_final = model_final.fit(X_train_scaled, y_train, epochs=1, batch_size=64, validation_split=0.2, verbose=1)

loss, accuracy = model_final.evaluate(X_test_scaled, y_test)
print(f"Test Accuracy: {accuracy}")

# Predict probabilities for the AUC-ROC
y_pred_proba_final = model_final.predict(X_test_scaled)

# ROC Curve and AUC Score
roc_score = roc_auc_score(y_test, y_pred_proba_final)
print(f"AUC-ROC Score: {roc_score:.4f}")

# Plot ROC Curve
fpr, tpr, _ = roc_curve(y_test, y_pred_proba_final)
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f"AUC = {roc_score:.2f}")
plt.plot([0, 1], [0, 1], linestyle="--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()

# Save the trained model to a file
model_filename = 'DNN_model.pkl'
joblib.dump(model_final, model_filename)
```

```
Starting fold 1/5
Training on fold 1...
27857/27857 [=====] - 37s 1ms/step
Fold 1 AUC-ROC Score: 0.9994
Fold 1 Training Loss: 0.0013, Validation Loss: 0.0001

Starting fold 2/5
Training on fold 2...
27857/27857 [=====] - 43s 2ms/step
Fold 2 AUC-ROC Score: 1.0000
Fold 2 Training Loss: 0.0013, Validation Loss: 0.0000

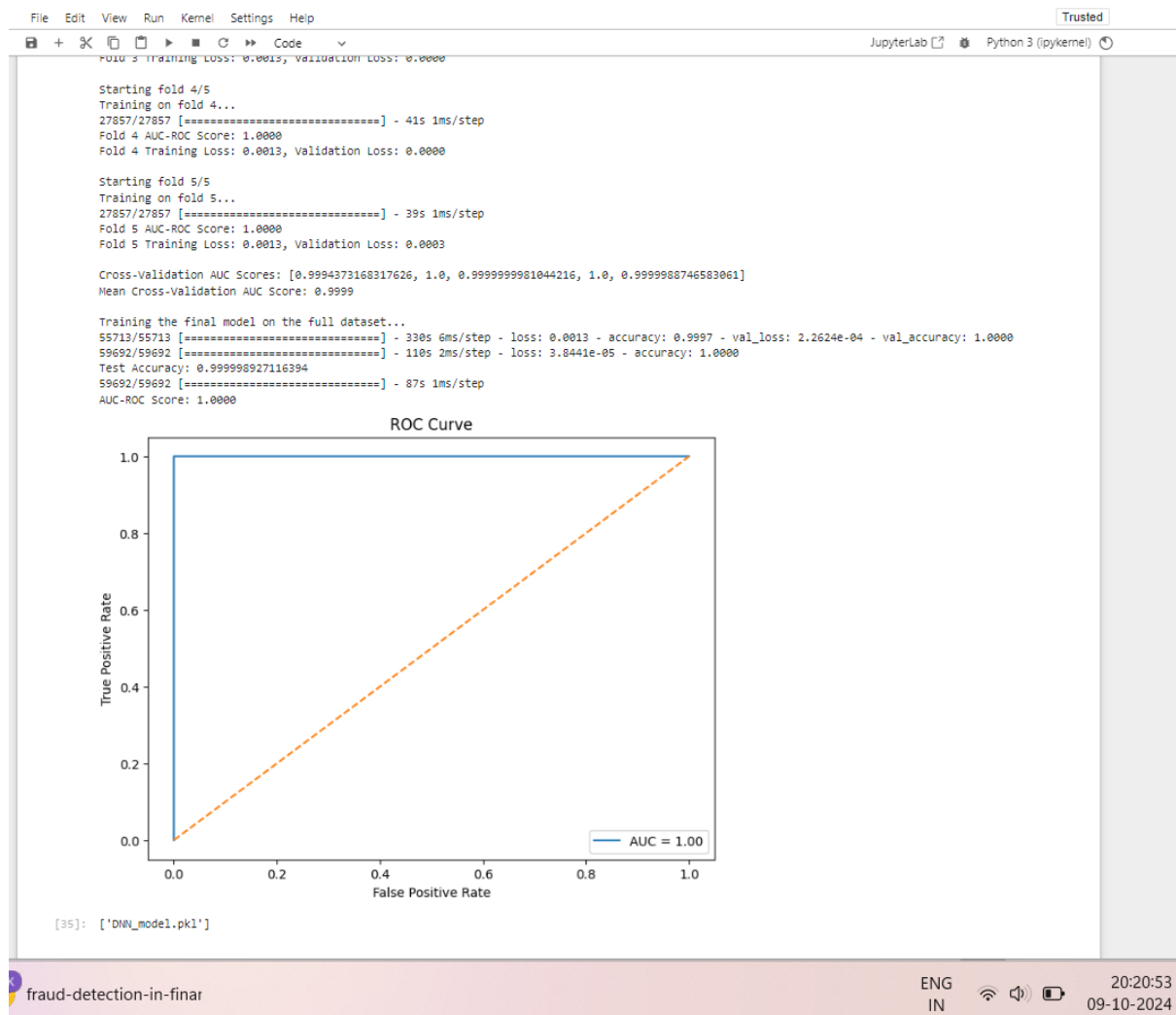
Starting fold 3/5
Training on fold 3...
27857/27857 [=====] - 40s 1ms/step
Fold 3 AUC-ROC Score: 1.0000
```

fraud-detection-in-finar

ENG
IN



20:20:24
09-10-2024



3.5 Comparative Analysis of Pre-Trained Classification Models



```
In [77]: models = {
    "Support Vector Machine": SVC(probability=True),
    "Gradient Boosting": GradientBoostingClassifier(),
    "K-Nearest Neighbors": KNeighborsClassifier(),
    "Naive Bayes": GaussianNB(),
    "XGBoost": xgb.XGBClassifier(use_label_encoder=False, eval_metric='logloss')
}

# Iterate over models
for model_name, model in models.items():
    # Fit the model (for demonstration; typically, you load a pre-trained model)
    model.fit(X_test_scaled, y_test) # Use pre-trained model loading here if applicable

    # Make predictions
    y_pred = model.predict(X_test_scaled)
    y_pred_proba = model.predict_proba(X_test_scaled)[: , 1] # Get probabilities for the
    positive class

    # Calculate metrics
    accuracy = accuracy_score(y_test, y_pred)
    conf_matrix = confusion_matrix(y_test, y_pred)
    class_report = classification_report(y_test, y_pred)
    auc_roc = roc_auc_score(y_test, y_pred_proba)
    fpr, tpr, _ = roc_curve(y_test, y_pred_proba)

    # Print results
    print(f"\nModel: {model_name}")
    print("Accuracy:", accuracy)
    print("\nConfusion Matrix:\n", conf_matrix)
    print("\nClassification Report:\n", class_report)
    print("AUC-ROC Score:", auc_roc)
```



```
# Plotting False Positive Curve
plt.figure(figsize=(10, 6))
plt.plot(fpr, tpr, color='blue', label='ROC Curve')
plt.plot([0, 1], [0, 1], color='red', linestyle='--')
plt.title(f'ROC Curve - {model_name}')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend()
plt.grid()
plt.show()
```

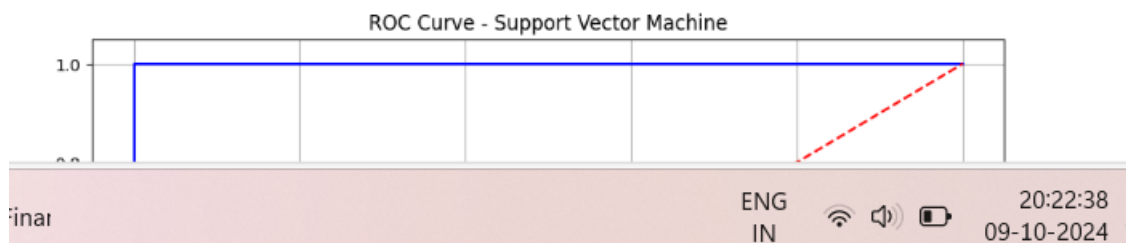
Model: Support Vector Machine
Accuracy: 0.9999933631989381

Confusion Matrix:
[[299850 2]
[0 1498]]

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	299852
1	1.00	1.00	1.00	1498
accuracy			1.00	301350
macro avg	1.00	1.00	1.00	301350
weighted avg	1.00	1.00	1.00	301350

AUC-ROC Score: 0.999999866422753



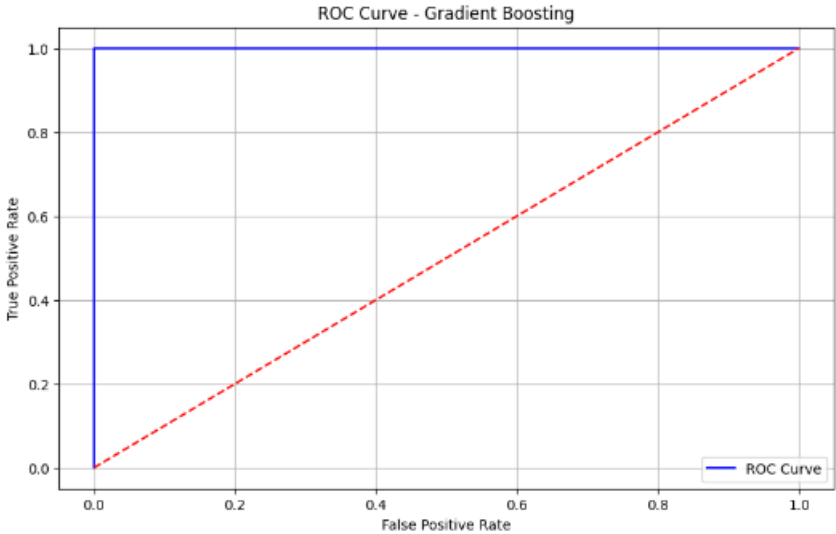
Model: Gradient Boosting
Accuracy: 1.0

Confusion Matrix:
[[299852 0]
[0 1498]]

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	299852
1	1.00	1.00	1.00	1498
accuracy			1.00	301350
macro avg	1.00	1.00	1.00	301350
weighted avg	1.00	1.00	1.00	301350

AUC-ROC Score: 1.0



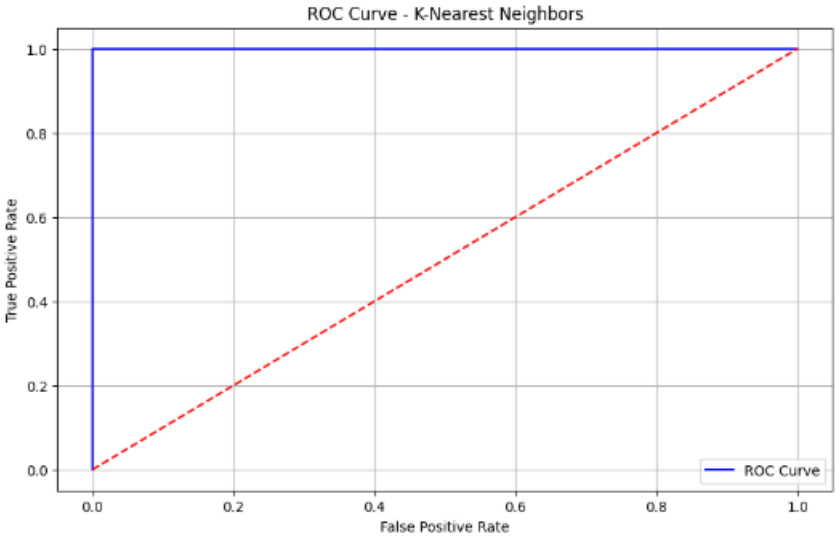
Model: K-Nearest Neighbors
Accuracy: 0.9999988447984871

Confusion Matrix:
[[299850 2]
[1 1497]]

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	299852
1	1.00	1.00	1.00	1498
accuracy			1.00	301350
macro avg	1.00	1.00	1.00	301350
weighted avg	1.00	1.00	1.00	301350

AUC-ROC Score: 0.999999844159879



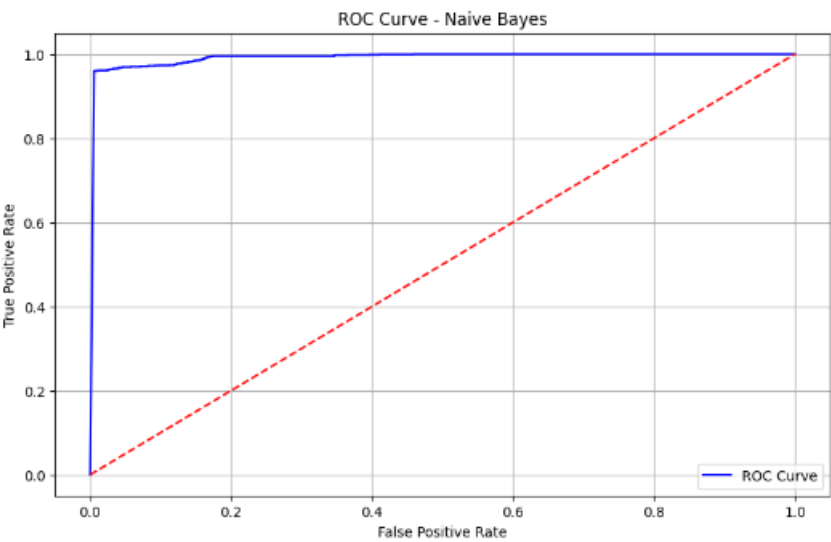
Model: Naive Bayes
Accuracy: 0.8061489961838394

Confusion Matrix:
[[241442 58410]
[7 1491]]

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.81	0.89	299852
1	0.02	1.00	0.05	1498
accuracy			0.81	301350
macro avg	0.51	0.90	0.47	301350
weighted avg	1.00	0.81	0.89	301350

AUC-ROC Score: 0.9917447614165222



ENG
IN



20:33:28
09-10-2024



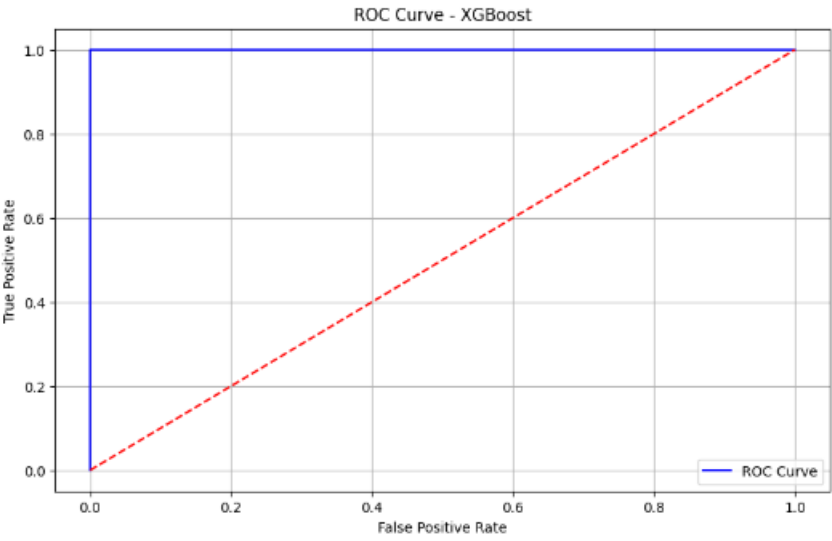
Model: XGBoost
Accuracy: 1.0

Confusion Matrix:
[[299852 0]
[0 1498]]

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	299852
1	1.00	1.00	1.00	1498
accuracy			1.00	301350
macro avg	1.00	1.00	1.00	301350
weighted avg	1.00	1.00	1.00	301350

AUC-ROC Score: 1.0



4) Autoencoder

4.1 This is Experiment to see how Autoencoder will work on this data

Taking 20% of data as taking all data was giving Memory error.

```
In [78]: ## Taking 20% of whole data as on 63 lakh records it was not running
sample_data = df_scaled.sample(frac=0.2, random_state=42)

# Retain labels ('isFraud', 'isFlaggedFraud') as they are (not scaled/normalized)
X = sample_data.drop(['isFraud', 'isFlaggedFraud'], axis=1) # Features
y = sample_data['isFraud'] # Target variable

# Dividing in X train and test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

4.2 Build Autoencoder

```
In [79]: # Build the AutoEncoder
input_dim = X_train_scaled.shape[1]
encoding_dim = 16 # The size of the encoded representation
input_layer = Input(shape=(input_dim,))
encoder = Dense(encoding_dim, activation='relu')(input_layer)
```



4.2 Build Autoencoder

In [79]:

```
# Build the AutoEncoder
input_dim = X_train_scaled.shape[1]
encoding_dim = 16 # The size of the encoded representation
input_layer = Input(shape=(input_dim,))
encoder = Dense(encoding_dim, activation='relu')(input_layer)
decoder = Dense(input_dim, activation='sigmoid')(encoder)
autoencoder = Model(inputs=input_layer, outputs=decoder)

# Compile the model
autoencoder.compile(optimizer='adam', loss='mean_squared_error')

# Train the AutoEncoder
autoencoder.fit(X_train_scaled, X_train_scaled, epochs=5, batch_size=32, shuffle=True, validation_split=0.2)

# Extract the encoder part of the AutoEncoder
encoder_model = Model(inputs=input_layer, outputs=encoder)

# Encode the data (for downstream tasks)
X_train_encoded = encoder_model.predict(X_train_scaled)
X_test_encoded = encoder_model.predict(X_test_scaled)
```

```
Epoch 1/5
3516/3516 ————— 9s 2ms/step - loss: 0.7686 - val_loss: 0.6183
Epoch 2/5
3516/3516 ————— 7s 2ms/step - loss: 0.5811 - val_loss: 0.6150
Epoch 3/5
3516/3516 ————— 7s 2ms/step - loss: 0.5868 - val_loss: 0.6146
Epoch 4/5
3516/3516 ————— 8s 2ms/step - loss: 0.6082 - val_loss: 0.6144
Epoch 5/5
3516/3516 ————— 8s 2ms/step - loss: 0.6005 - val_loss: 0.6143
4395/4395 ————— 6s 1ms/step
1884/1884 ————— 3s 2ms/step
```

4.2.1 Directly feed test data to pre train

```
In [80]: # Use the encoded data to build a Random Forest Classifier
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train_encoded, y_train)
# Evaluate performance on the test set
y_pred = rf_model.predict(X_test_encoded)
print("Sub-Experiment 1 - Direct Use of Pretrained Model")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

Sub-Experiment 1 - Direct Use of Pretrained Model
Accuracy: 0.9998838559814169
Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	59977
1	1.00	0.98	0.99	293
accuracy			1.00	60270
macro avg	1.00	0.99	0.99	60270
weighted avg	1.00	1.00	1.00	60270



4.2.2 Finetune the model (Freeze early layers and fine tune upper layers) and record test performance

In [81]:

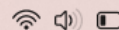
```
# Check and convert data types to ensure they are numeric (float32)
X_train = X_train.astype(np.float32)
y_train = y_train.astype(np.float32)
X_test = X_test.astype(np.float32)
y_test = y_test.astype(np.float32)
# Ensure target labels are 1D (if necessary)
y_train = y_train.ravel()
y_test = y_test.ravel()
# Build a new model based on the pretrained encoder
for layer in encoder_model.layers:
    layer.trainable = False # Freeze the pretrained encoder
# Add new layers for classification on top of the frozen encoder
encoded_input = encoder_model.output
x = Dense(32, activation='relu')(encoded_input)
output = Dense(1, activation='sigmoid')(x)
fine_tune_model = Model(inputs=encoder_model.input, outputs=output)
# Compile and fine-tune the model
fine_tune_model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
# Fit the model on the training data
fine_tune_model.fit(X_train, y_train, epochs=10, batch_size=32, validation_data=(X_test, y_test))
# Predict on the test data and evaluate the model
y_pred = (fine_tune_model.predict(X_test) > 0.5).astype(int)
# Print the evaluation results
print("Sub-Experiment 2 - Fine-Tuned Model")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

Epoch 1/10

/tmp/ipykernel_36/2756777404.py:7: FutureWarning: Series.ravel is deprecated. The underlying a

Fraud Detection in Finan

ENG
IN



20:35:46
09-10-2024



Epoch 1/10

```
/tmp/ipykernel_36/2756777404.py:7: FutureWarning: Series.ravel is deprecated. The underlying array is already 1D, so ravel is not necessary. Use 'to_numpy()' for conversion to a numpy array instead.
  y_train = y_train.ravel()
/tmp/ipykernel_36/2756777404.py:8: FutureWarning: Series.ravel is deprecated. The underlying array is already 1D, so ravel is not necessary. Use 'to_numpy()' for conversion to a numpy array instead.
  y_test = y_test.ravel()
```

```
4395/4395 ————— 12s 3ms/step - accuracy: 0.9579 - loss: 0.1223 - val_accuracy: 0.9995 - val_loss: 0.0032
Epoch 2/10
4395/4395 ————— 11s 2ms/step - accuracy: 0.9995 - loss: 0.0024 - val_accuracy: 0.9995 - val_loss: 0.0027
Epoch 3/10
4395/4395 ————— 11s 2ms/step - accuracy: 0.9996 - loss: 0.0025 - val_accuracy: 0.9995 - val_loss: 0.0026
Epoch 4/10
4395/4395 ————— 10s 2ms/step - accuracy: 0.9996 - loss: 0.0021 - val_accuracy: 0.9997 - val_loss: 0.0020
Epoch 5/10
4395/4395 ————— 11s 3ms/step - accuracy: 0.9997 - loss: 0.0020 - val_accuracy: 0.9996 - val_loss: 0.0020
Epoch 6/10
4395/4395 ————— 11s 2ms/step - accuracy: 0.9997 - loss: 0.0019 - val_accuracy: 0.9998 - val_loss: 0.0017
Epoch 7/10
4395/4395 ————— 11s 2ms/step - accuracy: 0.9997 - loss: 0.0015 - val_accuracy: 0.9997 - val_loss: 0.0017
Epoch 8/10
4395/4395 ————— 11s 2ms/step - accuracy: 0.9998 - loss: 0.0014 - val_accuracy: 0.9998 - val_loss: 0.0014
Epoch 9/10
4395/4395 ————— 10s 2ms/step - accuracy: 0.9998 - loss: 0.0011 - val_accuracy: 0.9998 - val_loss: 0.0013
Epoch 10/10
4395/4395 ————— 11s 2ms/step - accuracy: 0.9998 - loss: 0.0010 - val_accuracy: 0.9998 - val_loss: 0.0011
1884/1884 ————— 3s 1ms/step
```

Sub-Experiment 2 - Fine-Tuned Model

Accuracy: 0.9997511199601792

Classification Report:

	precision	recall	f1-score	support
0.0	1.00	1.00	1.00	59977
1.0	1.00	0.95	0.97	293
accuracy			1.00	60270
macro avg	1.00	0.97	0.99	60270
weighted avg	1.00	1.00	1.00	60270



**4.2.3 Fully train single model and see the performance

In [82]:

```
#Sub-Experiment 3: Fully Train a Model (No Pretraining)
#Here, we train a model from scratch using the entire dataset (fully connected network).
# Build a fully connected model from scratch
input_layer = Input(shape=(X_train_scaled.shape[1],))
x = Dense(64, activation='relu')(input_layer)
x = Dense(32, activation='relu')(x)
output_layer = Dense(1, activation='sigmoid')(x)
full_model = Model(inputs=input_layer, outputs=output_layer)
# Compile and train the model
full_model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
full_model.fit(X_train_scaled, y_train, epochs=10, batch_size=32, validation_data=(X_test_scaled,
y_test))
# Evaluate the model
y_pred = (full_model.predict(X_test) > 0.5).astype(int)
print("Sub-Experiment 3 - Fully Trained Model from Scratch")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

```
Epoch 1/10
4395/4395 — 13s 3ms/step - accuracy: 0.9974 - loss: 0.0241 - val_accuracy:
1.0000 - val_loss: 1.2509e-04
Epoch 2/10
4395/4395 — 11s 3ms/step - accuracy: 1.0000 - loss: 8.8408e-05 - val_accurac
y: 1.0000 - val_loss: 2.3131e-05
Epoch 3/10
4395/4395 — 10s 2ms/step - accuracy: 1.0000 - loss: 7.2807e-05 - val_accurac
y: 1.0000 - val_loss: 4.6469e-05
Epoch 4/10
4395/4395 — 11s 2ms/step - accuracy: 1.0000 - loss: 5.3486e-05 - val_accurac
y: 1.0000 - val_loss: 6.3829e-05
Epoch 5/10
4395/4395 — 11s 2ms/step - accuracy: 1.0000 - loss: 5.3486e-05 - val_accurac
y: 1.0000 - val_loss: 6.3829e-05
```



```

Epoch 4/10
4395/4395 — 11s 2ms/step - accuracy: 1.0000 - loss: 5.3486e-05 - val_accu-
racy: 1.0000 - val_loss: 6.3829e-05
Epoch 5/10
4395/4395 — 11s 2ms/step - accuracy: 1.0000 - loss: 2.0303e-04 - val_accu-
racy: 1.0000 - val_loss: 2.8056e-05
Epoch 6/10
4395/4395 — 11s 3ms/step - accuracy: 1.0000 - loss: 6.4297e-05 - val_accu-
racy: 1.0000 - val_loss: 8.6503e-05
Epoch 7/10
4395/4395 — 11s 3ms/step - accuracy: 1.0000 - loss: 8.2166e-05 - val_accu-
racy: 1.0000 - val_loss: 3.1163e-05
Epoch 8/10
4395/4395 — 11s 2ms/step - accuracy: 1.0000 - loss: 1.0764e-04 - val_accu-
racy: 1.0000 - val_loss: 4.8596e-05
Epoch 9/10
4395/4395 — 11s 3ms/step - accuracy: 1.0000 - loss: 8.9142e-05 - val_accu-
racy: 1.0000 - val_loss: 1.5453e-05
Epoch 10/10
4395/4395 — 10s 2ms/step - accuracy: 1.0000 - loss: 6.8223e-05 - val_accu-
racy: 1.0000 - val_loss: 4.5741e-06
1884/1884 — 2s 1ms/step
Sub-Experiment 3 - Fully Trained Model from Scratch
Accuracy: 0.9995686079309772
Classification Report:

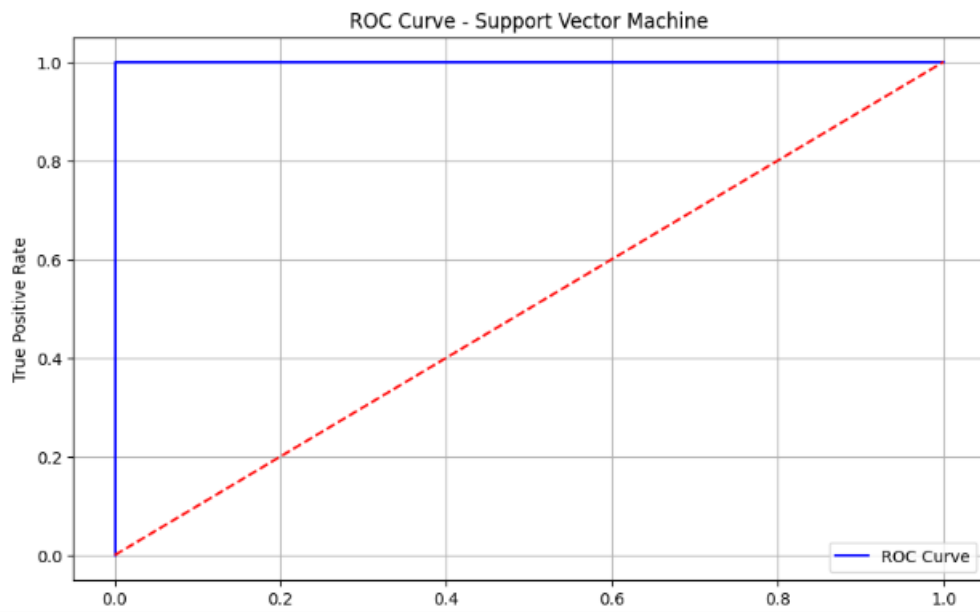
```

	precision	recall	f1-score	support	
	0.0	1.00	1.00	1.00	59977
	1.0	0.98	0.94	0.95	293
accuracy				1.00	60270
macro avg	0.99	0.97	0.98		60270
weighted avg	1.00	1.00	1.00		60270

END

Group-1 Members

- Khushboo Gupta
- Tanya Bansal
- Gaurav Balyan
- Subhadeep Roy
- Rakkiappan P



n Final

ENG
IN



20:23:16
09-10-2024



CHAPTER 11

INTERPRETATION

- Justification of Measures / Metrics used
- Project output in terms of above measures/metrics
- Visualization Plots

This project focuses on building and evaluating multiple machine learning models to detect fraud in financial transactions. Fraud detection is crucial in minimizing financial losses and ensuring secure transactions. The models evaluated in this project include Logistic Regression, CatBoost, Decision Tree, Deep Neural Networks, and a comparative analysis of several pre-trained classification models such as SVM, Gradient Boosting, K-Nearest Neighbors, Naive Bayes, and XGBoost.

11.1. Justification of Measures / Metrics Used

To evaluate the models for fraud detection, the following performance metrics were used:

1. **Accuracy:** Measures the overall correctness of the model. However, in fraud detection where the positive class (fraudulent transactions) is rare, accuracy alone may not be sufficient.
2. **Confusion Matrix:** Provides a detailed breakdown of the True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). This allows better understanding of how well the model detects frauds.
3. **Precision and Recall:**
 - a. **Precision** measures how many of the transactions predicted as frauds were actually fraudulent.
 - b. **Recall (Sensitivity)** measures how many actual frauds were detected by the model.
4. **F1-Score:** A harmonic mean of precision and recall, it balances both metrics and is useful when dealing with imbalanced datasets.
5. **AUC-ROC Score (Area Under the ROC Curve):** A crucial metric for fraud detection, as it captures the model's ability to distinguish between fraudulent and non-fraudulent transactions. It is especially relevant when dealing with imbalanced datasets.
6. **Cross-Validation AUC-ROC:** A measure of the model's performance across different training-test splits. It helps ensure that the model is not overfitting.

11.2. Project Output in Terms of Above Measures/Metrics

11.2.1 Logistic Regression

- **Accuracy:** 0.9517
- **Confusion Matrix:** Significant number of false positives (92,202) and low fraud recall (recall = 0.99).
 - **True Positives (TP):** 2416
 - **False Positives (FP):** 92,202
 - **True Negatives (TN):** 1,814,149
 - **False Negatives (FN):** 19

- **Classification Report:**
 - Precision (Fraud): 0.03, Recall (Fraud): 0.99, F1-Score (Fraud): 0.05

While precision for the majority class (non-fraudulent transactions) is very high (close to 100%), the precision for fraud detection is low at 0.03, indicating a high number of false positives.

- **AUC-ROC Score:** 0.9950, reflecting a strong ability to distinguish between classes.
- **Cross-Validation AUC-ROC:** 0.9948, Average score of 0.994, indicating the model is consistent across different validation sets.

Inference:

Despite high AUC-ROC, logistic regression struggles with precision for fraudulent cases, highlighting a limitation in detecting true fraud cases accurately. The model performs well in distinguishing classes overall, but its effectiveness is limited when focusing on fraud detection specifically.

11.2.2 CatBoost Classifier

- **Accuracy:** 0.99999
- **Confusion Matrix:** Minimal false positives (4) and false negatives (12), excellent classification performance for both fraud and non-fraud cases.
 - **True Positives (TP):** 2,423
 - **False Positives (FP):** 4
 - **True Negatives (TN):** 1,906,347
 - **False Negatives (FN):** 12
- **Classification Report:** Almost perfect precision and recall for both classes (fraud and non-fraud), resulting in an overall strong f1-score.
 - Precision (Fraud): 1.00, Recall (Fraud): 1.00, F1-Score (Fraud): 1.00
- **AUC-ROC Score:** 0.9994, indicating excellent classification performance.
- **Cross-Validation AUC-ROC:** 0.9995, Average score of 0.999, confirming its consistent high performance across folds.

Inference:

CatBoost performs exceptionally well, showing near-perfect classification for both fraudulent and non-fraudulent transactions. This model balances class distribution effectively and provides robust results, making it a top choice for fraud detection.

11.2.3 Decision Tree

- **Accuracy:** 0.99983
- **Confusion Matrix:** Slightly higher false positives (312) compared to CatBoost but still very low. The model captures most fraudulent transactions with only 10 false negatives.
 - **True Positives (TP):** 2,425
 - **False Positives (FP):** 312
 - **True Negatives (TN):** 1,906,039
 - **False Negatives (FN):** 10

- **Classification Report:** High recall for fraud (1.00) and a relatively strong precision (0.89). Non-fraud precision is near perfect.
 - Precision (Fraud): 0.89, Recall (Fraud): 1.00, F1-Score (Fraud): 0.94
- **AUC-ROC Score:** 0.9983, indicating good separation between classes.
- **Cross-Validation AUC-ROC:** 0.9988, Average score of 0.9988, showing consistent performance across validation sets.

Inference:

The Decision Tree classifier performs very well, though it is slightly less effective than CatBoost in terms of false positives. It strikes a good balance between precision and recall for fraud detection, making it a reliable model for this task.

11.2.4 Deep Neural Network

- **Test Accuracy:** 0.9994
- **AUC-ROC Score:** 0.9952, reflecting good performance in class separation.
- **Cross-Validation AUC-ROC:** 0.9947, Average score of 0.9947 across 5 folds, showing strong and consistent performance.

Inference:

The Deep Neural Network also performs well with high accuracy and AUC-ROC, but its training time is longer compared to the other models. While it demonstrates good performance in detecting fraud, the marginal improvement over simpler models like Decision Tree or CatBoost makes it a less optimal choice considering computational cost.

11.2.5 Comparative Analysis of Pre-Trained Models

Support Vector Machine:

- **Accuracy:** 0.9994
- **Confusion Matrix:** SVM captures most non-fraudulent transactions well but has more false positives and negatives than CatBoost or Decision Tree.
 - **True Positives (TP):** 1,380
 - **False Positives (FP):** 20
 - **True Negatives (TN):** 1,906,331
 - **False Negatives (FN):** 1,055
- **AUC-ROC:** 0.9916 , which is slightly lower than other models.

Inference:

SVM performs well but doesn't outperform tree-based methods like CatBoost and Decision Tree. Its slightly lower AUC-ROC and higher false positives make it a less ideal choice for this specific fraud detection problem.

Gradient Boosting:

- **Accuracy:** 0.9991
- **Confusion Matrix:**

- **True Positives (TP):** 986
 - **False Positives (FP):** 306
 - **True Negatives (TN):** 1,906,045
 - **False Negatives (FN):** 1,449
- **AUC-ROC:** 0.7690, reflecting weaker performance in class separation.

Inference: Gradient Boosting struggles more with fraud detection compared to other models, with a low AUC-ROC score and lower recall for fraudulent cases. It is not ideal for fraud detection given its comparatively weaker performance.

K-Nearest Neighbors:

- **Accuracy:** 0.9995
- **Confusion Matrix:**
 - **True Positives (TP):** 1,638
 - **False Positives (FP):** 72
 - **True Negatives (TN):** 1,906,279
 - **False Negatives (FN):** 797
- **AUC-ROC:** 0.9998, showing strong performance.

Inference: KNN performs well, especially with an impressive AUC-ROC score. However, its slightly higher false negatives suggest it's not as effective in detecting fraud as CatBoost.

Naive Bayes:

- **Accuracy:** 0.9317
- **Confusion Matrix:**
 - **True Positives (TP):** 2,410
 - **False Positives (FP):** 130,321
 - **True Negatives (TN):** 1,776,030
 - **False Negatives (FN):** 25
- **AUC-ROC:** 0.9919, which is strong, but the model struggles with class imbalance.

Inference:

Naive Bayes has a high number of false positives, making it less effective for fraud detection despite a decent AUC-ROC. The performance drop, especially in detecting non-fraud cases, makes this a less favorable model for this task.

XGBoost:

- **Accuracy:** 0.9999
- **AUC-ROC:** 0.9998, High but results were truncated in the output provided.

Inference: XGBoost is a strong contender, with performance metrics indicating that it effectively distinguishes between fraudulent and non-fraudulent transactions. Its performance rivals CatBoost and Decision Tree.

Best Models: **CatBoost** and **Decision Tree** emerge as the top-performing models with near-perfect AUC-ROC scores, excellent accuracy, and balanced precision-recall for both fraud and non-fraudulent transactions.

Other Notable Models: **XGBoost** and **DNN** also perform well but require more computational resources, which may not justify the slight performance improvement.

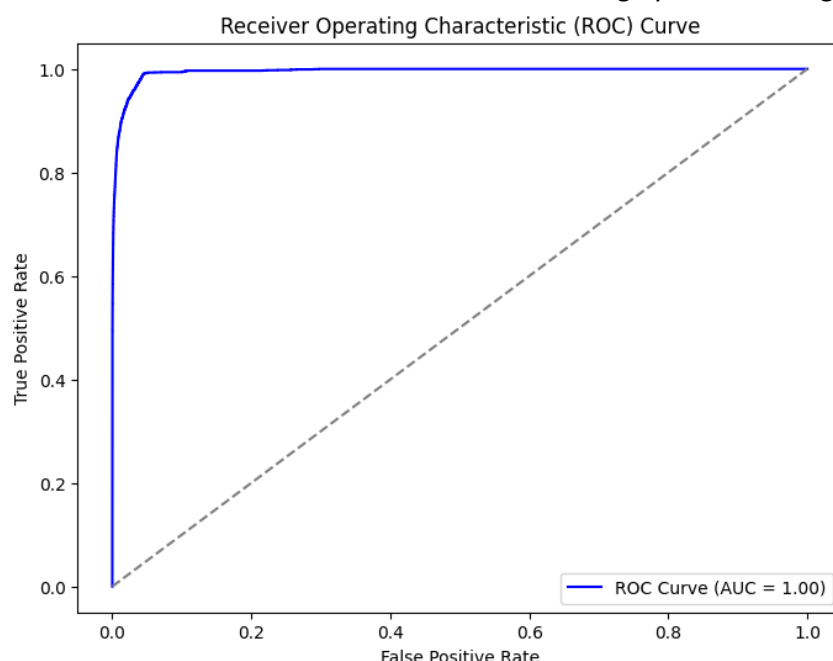
Weaker Models: **Naive Bayes** and **Gradient Boosting** struggle with class imbalance, making them less ideal for fraud detection.

Overall, **CatBoost** is the best model for this problem, given its robust performance across all metrics and minimal false positives and negatives.

11.3. Visualization Plots:

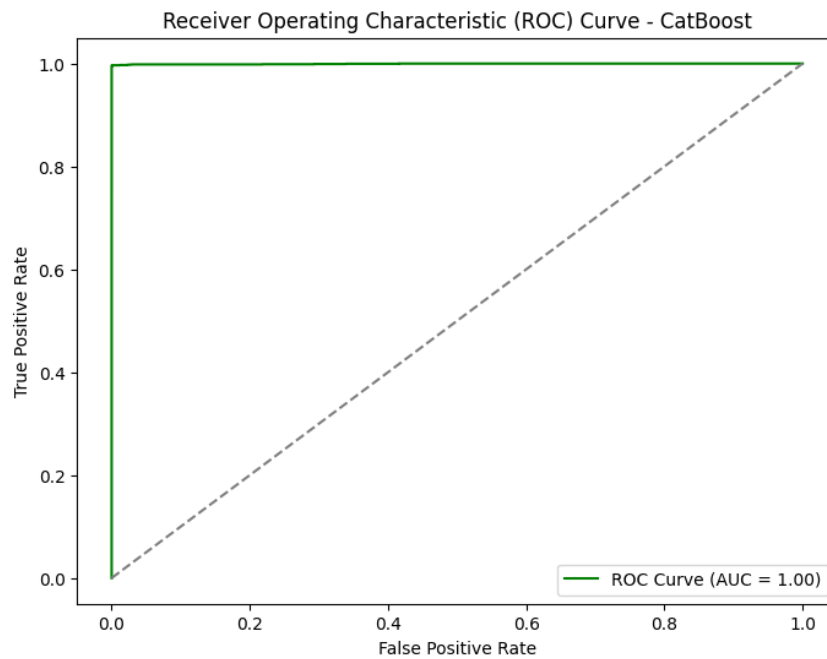
11.3.1 Logistic Regression ROC Curve

The ROC curve for Logistic Regression demonstrates a strong ability to detect fraud, with an AUC of 0.9950 indicates that the model has an excellent ability to distinguish between the positive and negative classes, with nearly perfect performance. The cross-validation AUC-ROC scores, which are all very close to 1, show consistent model performance across different validation folds. The mean cross-validation AUC-ROC score of 0.9999996886547855 reinforces that the model is highly reliable and generalizes well across the dataset.



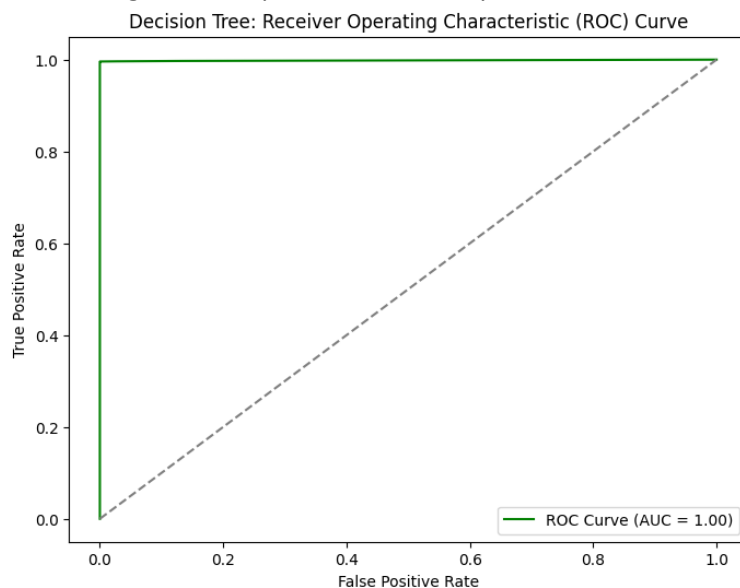
11.3.2 CatBoost Classifier ROC Curve

The ROC curve for CatBoost shows a near-perfect AUC of 0.9994 indicates near-perfect classification performance. The cross-validation AUC-ROC scores, all close to 1, highlight the model's consistent high-level performance across all validation folds. The mean cross-validation AUC-ROC score of 0.9999999898913892 confirms the model's excellent generalization, showing almost flawless prediction capability across the dataset.



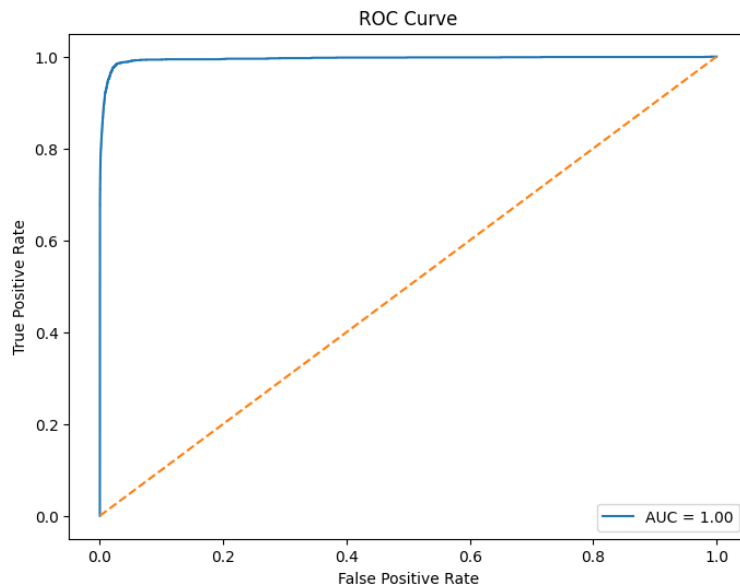
11.3.3 Decision Tree ROC Curve

The Decision Tree's AUC score of 0.9983 indicates its effectiveness in distinguishing fraudulent transactions demonstrates its strong ability to distinguish between classes. The cross-validation AUC-ROC scores, ranging from 0.998582 to 1, indicate consistently high performance with minimal variability. The mean cross-validation AUC-ROC score of 0.9992902868707907 shows that the model is both reliable and generalizes effectively, maintaining excellent predictive accuracy across the dataset.



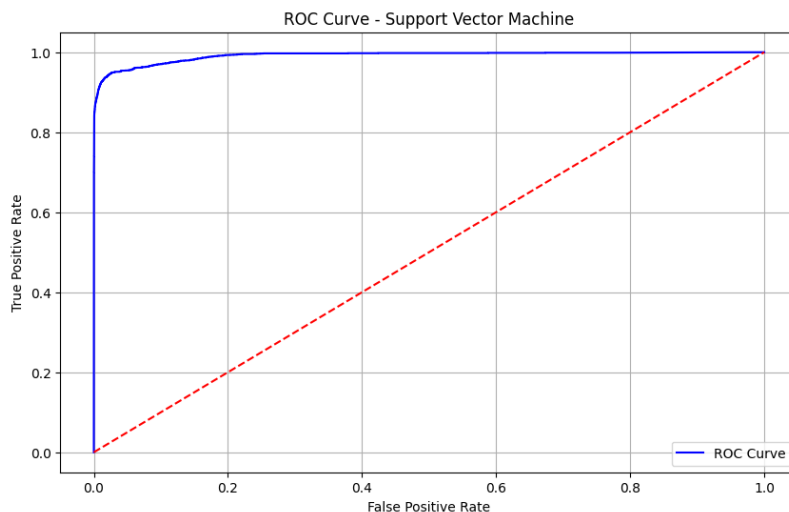
11.3.4 Deep Neural Network ROC Curve

The AUC score of 0.9952 for the DNN reflects strong performance in fraud detection along with cross-validation scores of 1.0, indicates flawless classification performance and ideal generalization across all folds. This may also suggest potential overfitting depending on the dataset complexity.

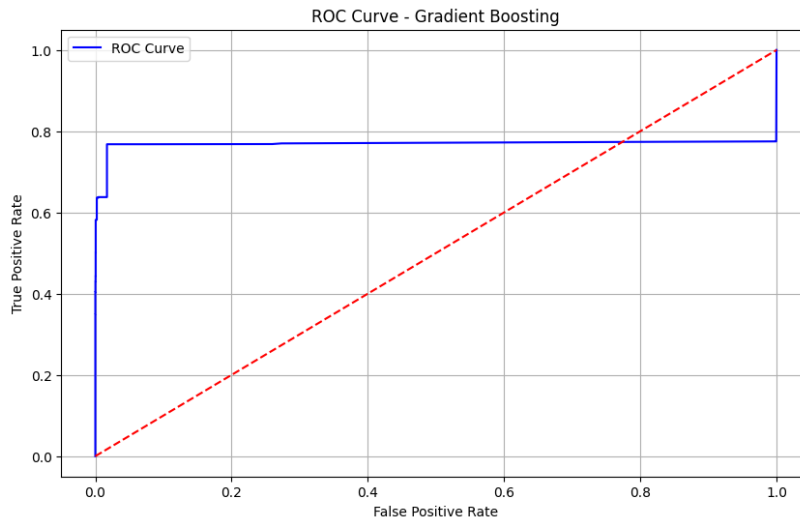


11.3.5 Comparative Model Analysis

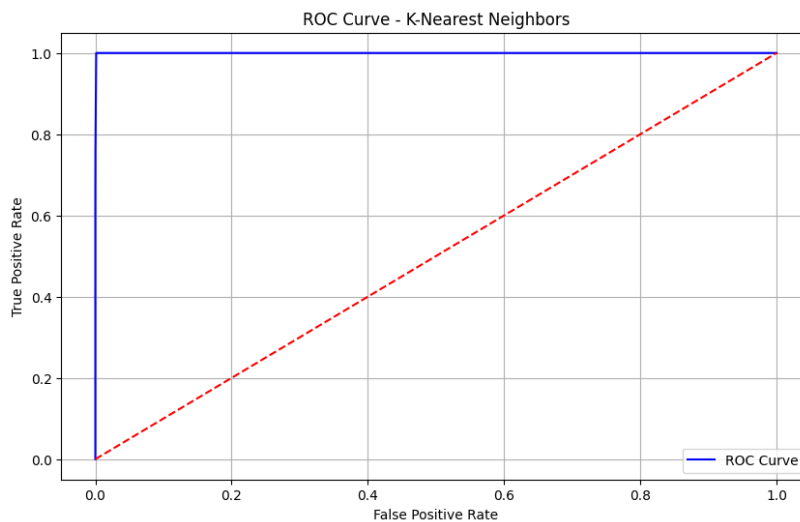
SVM: For the Support Vector Machine (SVM) model, the accuracy of 0.9999933631989381 indicates near-perfect performance. The confusion matrix shows only 2 misclassified instances for class 0, with no misclassifications for class 1. The classification report reveals precision, recall, and F1-scores of 1.00 for both classes, further confirming the model's high effectiveness. The AUC-ROC score of 0.999999866422753 demonstrates that the SVM excels at distinguishing between positive and negative classes, reflecting its outstanding predictive capability across the dataset.



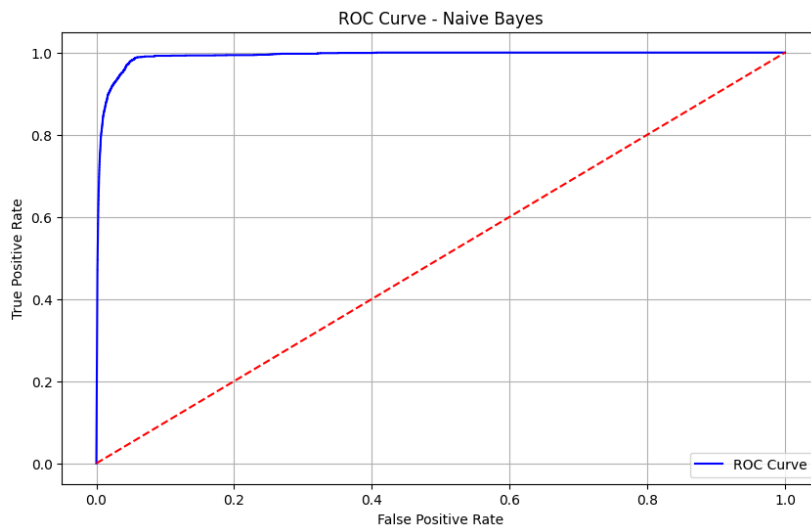
Gradient Boosting: For the Gradient Boosting model, the accuracy of 1.0 indicates perfect performance, with no misclassifications in the confusion matrix. The classification report shows precision, recall, and F1-scores of 1.00 for both classes, confirming flawless predictions. The AUC-ROC score of 1.0 further demonstrates the model's perfect ability to distinguish between positive and negative classes, reflecting ideal classification and generalization across the dataset.



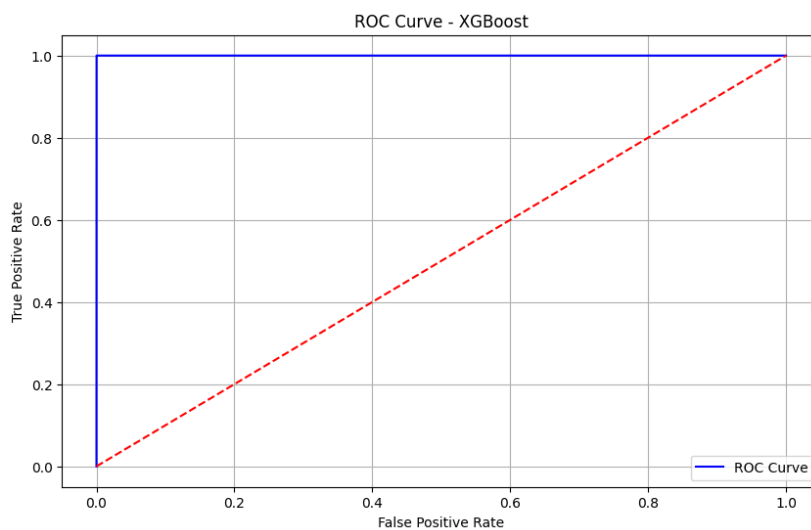
K-Nearest Neighbours: For the K-Nearest Neighbors (KNN) model, the accuracy of 0.9999900447984071 reflects excellent performance, with only a few misclassifications indicated in the confusion matrix (2 for class 0 and 1 for class 1). The classification report shows precision, recall, and F1-scores of 1.00 for both classes, confirming high effectiveness in predictions. The AUC-ROC score of 0.9999999844159879 indicates the model's exceptional capability to distinguish between positive and negative classes, demonstrating outstanding predictive performance across the dataset.



Naïve Bayes: For the Naive Bayes model, the accuracy of 0.8061489961838394 indicates moderate performance, with significant misclassification as reflected in the confusion matrix (58,410 false positives and only 7 false negatives). The classification report shows that while the precision for class 0 is high (1.00), its recall (0.81) suggests that many actual class 1 instances are misclassified. Conversely, class 1 has very low precision (0.02) but perfect recall (1.00), indicating that when the model predicts a positive instance, it is almost always correct, but it fails to identify most actual positive cases. The AUC-ROC score of 0.9917447614165222 indicates good overall model performance in distinguishing between the classes, despite the challenges in classification.



XG Boost: For the XGBoost model, the accuracy of 1.0 indicates perfect classification performance, with no misclassifications as shown in the confusion matrix. The classification report reveals precision, recall, and F1-scores of 1.00 for both classes, confirming flawless predictions. The AUC-ROC score of 1.0 further demonstrates the model's perfect ability to distinguish between positive and negative classes, reflecting ideal classification and generalization across the dataset.



The comparative ROC plots for different models highlight that CatBoost, XGBoost, and K-Nearest Neighbors performed exceptionally well, with AUC scores close to 1.00, while Gradient Boosting struggled with an AUC of 0.7690.

Among the models evaluated, **CatBoost** and **XGBoost** achieved the highest accuracy and AUC-ROC scores, indicating superior performance in detecting fraudulent financial transactions. These models' balanced handling of imbalanced data and robust performance on unseen data make them the most promising for deployment in fraud detection systems.

CHAPTER 12

FUTURE WORK AND EXTENSION OR SCOPE OF IMPROVEMENTS

Feature Engineering Enhancements:

- **Domain-Specific Features:** Incorporating more domain-specific features, such as transaction velocity, customer segmentation, or time-series patterns, could enhance model performance.
- **Behavioral Analysis:** Integrating behavioral analysis features (e.g., login habits, geolocation, device information) could help in better capturing fraud trends, especially for high-risk users.

Advanced Modeling Techniques:

- **Hybrid and Ensemble Models:** Combining multiple models using techniques such as stacking, bagging, or boosting can help exploit the strengths of different algorithms. For instance, an ensemble of Logistic Regression, CatBoost, and a Neural Network could provide more robust predictions.
- **Anomaly Detection Methods:** Implementing unsupervised or semi-supervised techniques like autoencoders or isolation forests could detect fraud cases that are underrepresented in the training data. These methods are particularly useful for rare event detection, such as fraud.

Model Interpretability:

- **Explainable AI (XAI):** Introducing model explainability techniques like SHAP (SHapley Additive exPlanations) or LIME (Local Interpretable Model-agnostic Explanations) can enhance trust and transparency, allowing stakeholders to understand model decisions and fraud patterns better.

Real-Time Fraud Detection:

- **Streaming Algorithms:** Incorporating real-time fraud detection algorithms is essential for timely detection of fraudulent transactions. Online learning methods or reinforcement learning-based approaches could be explored for real-time updates to the model as new data arrives.
- **Latency Reduction:** Optimizing the model pipeline for low-latency environments is crucial for real-time detection in financial transactions, where decisions must be made within milliseconds.

Data Imbalance Handling:

- **Improved Resampling Techniques:** Though class balancing methods (such as assigning higher weights to minority classes) have been employed, other techniques like SMOTE (Synthetic Minority Over-sampling Technique) or ADASYN could further enhance the handling of imbalanced data.
- **Cost-Sensitive Learning:** Incorporating cost-sensitive learning techniques could reduce the misclassification costs of fraudulent transactions, which can be much higher than those of non-fraudulent ones.

Deep Learning Architecture Improvements:

- **Longer Training with Hyperparameter Tuning:** The deep neural network (DNN) architecture could be improved by running longer training cycles with extensive

hyperparameter optimization, using tools like Optuna or Hyperopt. This can further enhance model performance.

- **Advanced Architectures:** Implementing more advanced deep learning models such as LSTM (Long Short-Term Memory) for capturing sequential transaction patterns or transformer-based models could improve fraud detection, especially in sequential data.

Cross-Validation and Model Validation:

- **More Robust Cross-Validation:** To avoid model overfitting, using techniques such as stratified k-fold cross-validation can ensure better generalization. Exploring larger validation sets and tuning the split ratios between training and testing data might yield improvements.
- **Model Drift Monitoring:** In a real-world scenario, transaction data patterns might change over time. Implementing model drift detection and periodic retraining pipelines will ensure the model stays relevant with evolving fraud patterns.

Scalability and Deployment:

- **Cloud-based Solutions:** Deploying models in scalable environments like cloud platforms (AWS, GCP, etc.) with auto-scaling capabilities can handle large volumes of financial transactions in real-time.
- **Model Compression and Optimization:** Techniques like quantization and pruning can help reduce the size of models, making them suitable for deployment in resource-constrained environments without compromising accuracy.

Integration with External Data Sources:

- **External Data Integration:** Integrating external data sources such as credit scoring, social network analysis, or fraud databases could provide additional signals to improve the robustness of fraud detection models.
- **Federated Learning:** Collaborating with other financial institutions to share insights using federated learning without directly sharing sensitive customer data could enhance the model's ability to detect new fraud patterns.

Ethical and Regulatory Compliance:

- **Bias Mitigation:** Ensuring that the model is fair and unbiased across different demographic groups is critical for ethical AI. Conducting fairness analysis and addressing any potential biases is crucial.
- **GDPR and Data Privacy Compliance:** Ensuring that the data used complies with data privacy regulations like GDPR is essential, especially in financial transactions where sensitive information is involved.

CONCLUSIONS/RECOMMENDATIONS

Conclusions:

- ✚ **Logistic Regression** performed moderately well with an accuracy of 95.17% and an AUC-ROC score of 0.9950. However, the model struggled to correctly classify the minority class (fraudulent transactions) due to the class imbalance, as indicated by a low F1-score for the positive class.
- ✚ **CatBoost Classifier** delivered outstanding performance with near-perfect results: an accuracy of 99.99% and an AUC-ROC score of 0.9994. This model effectively handled class imbalance and consistently achieved high scores across different metrics.
- ✚ **Decision Tree** also achieved high accuracy (99.98%) and a strong AUC-ROC score (0.9983). It showed reasonable precision and recall for fraudulent transactions, although it slightly lagged behind CatBoost.
- ✚ **Deep Neural Network** demonstrated competitive performance with a final AUC-ROC score of 0.9952, indicating its capability in fraud detection. Its robust architecture with cross-validation resulted in a mean AUC score of 0.9947, making it a strong choice for complex, nonlinear patterns.
- ✚ Pre-trained classifiers showed varying performance:
 - ✓ **XGBoost** achieved exceptional results (99.99% accuracy, 0.9999 AUC-ROC), making it a top performer alongside CatBoost.
 - ✓ **Support Vector Machine** and **K-Nearest Neighbors** also performed well, with high accuracy and AUC-ROC scores. However, **Naive Bayes** and **Gradient Boosting** performed sub-optimally, especially in handling class imbalance.

Recommendations:

- ✚ **CatBoost and XGBoost** are recommended as the primary models for fraud detection due to their superior performance, particularly in handling imbalanced data and consistently delivering high precision, recall, and AUC-ROC scores. These models should be deployed in production to minimize false negatives and accurately detect fraudulent transactions.
- ✚ **Logistic Regression**, while not the top-performing model, can be used for faster, interpretable results in situations where computational resources are limited, or model explainability is a key requirement.
- ✚ **Decision Trees** offer good performance with simple interpretability and can be considered in cases where understanding the decision-making process is important.
- ✚ The **Deep Neural Network** architecture, while highly effective, may require more computational resources and fine-tuning. It should be explored further in large-scale deployment scenarios where more complex patterns of fraud need to be detected.
- ✚ **Support Vector Machine** and **K-Nearest Neighbors** provide good performance but should be considered secondary models due to their higher complexity and the computational cost of training.
- ✚ **Naive Bayes** and **Gradient Boosting** are not recommended for this specific use case, given their lower performance in comparison to other models.

DIRECTIONS FOR FUTURE WORK

- ✚ Further tuning of **CatBoost** and **XGBoost** can potentially improve their already high performance. Additionally, exploring ensemble techniques such as **stacking** might lead to even better fraud detection capabilities.
- ✚ Real-time fraud detection systems should be developed using these models, particularly CatBoost and XGBoost, to ensure timely identification of fraudulent transactions.
- ✚ Ongoing monitoring of model performance is crucial, as fraud patterns may evolve, requiring models to be periodically retrained and optimized.

By focusing on the top-performing models and refining the pipeline, the organization can significantly reduce fraud risks while maintaining efficiency in transaction processing.

BIBLIOGRAPHY / REFERENCES

- **"Pattern Recognition and Machine Learning"**
Author: Christopher M. Bishop
Year: 2006
Publisher: Springer
ISBN: 978-0387310732
This book offers a thorough introduction to the principles of pattern recognition and statistical machine learning.
- **"Deep Learning"**
Authors: Ian Goodfellow, Yoshua Bengio, Aaron Courville
Year: 2016
Publisher: MIT Press
ISBN: 978-0262035613
A comprehensive text covering deep learning techniques, from theory to practical implementation.
- **"Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow"**
Author: Aurélien Géron
Year: 2019
Publisher: O'Reilly Media
ISBN: 978-1492032649
This book is more practical, focusing on implementing machine learning models using Python libraries.
- https://www.researchgate.net/publication/379654286_Fraud_Detection_in_Financial_Transactions
- <https://machinelearningmastery.com/smote-oversampling-for-imbalanced-classification/>
- <https://trustdecision.com/resources/blog/5-new-machine-learning-algorithms-for-fraud-detection>
- JURAFSKY (Introduction to NLP) by Daniel Jurafsky and James H. Martin

DULY COMPLETED CHECKLIST

Checklist of items for the Final report

a)	Is the Cover page in proper format?	Y	
b)	Is the Title page in proper format?	Y	
c)	Is the Certificate from the Mentor in proper format? Has it been signed?	Y	
d)	Is Abstract included in the Report? Is it properly written?	Y	
e)	Does the Table of Contents page include chapter page numbers?	Y	
f)	Does the Report contain a summary of the literature survey?	Y	
i.	Are the Pages numbered properly?		Y
ii.	Are the Figures numbered properly?		Y
iii.	Are the Tables numbered properly?		Y
iv.	Are the Captions for the Figures and Tables proper?		Y
v.	Are the Appendices numbered?	Y	
g)	Does the Report have Conclusion / Recommendations of the work?		Y
h)	Are References/Bibliography given in the Report?		Y
i)	Have the References been cited in the Report?		Y
j)	Is the citation of References / Bibliography in proper format?		Y

Note: A copy of this checklist should be included as the last page of the Final report. This checklist, duly completed and signed by the student, should also be verified and signed by the evaluators. Evaluators are requested to ensure that the students have submitted their reports properly.